SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

2026 **BC. Marcel Soták**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav informatiky a matematiky (FEI)
Školiteľ:	Ing. Maroš Čavojský, PhD.

2026 **BC. Marcel Soták**

Slovenská technická univerzita v Bratislave
Ústav jadrového a fyzikálneho inžinierstva

Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025
Evidenčné číslo: FEI-XXXXX-YYYYYY

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Meno Priezvisko**
ID študenta:	012345
Študijný program:	jadrové a fyzikálne inžinierstvo
Študijný odbor:	elektrotechnika
Vedúci práce:	tituly, Meno Priezvisko, tituly
Vedúci pracoviska:	tituly Meno Priezvisko, tituly
Miesto vypracovania:	Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

1. Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
2. Napíšte návod na písanie záverečnej práce s použitím vytvorenej šablóny.
3. Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

1. Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf
2. Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecnych-prac-ich-bibliografickej-registracii-uchovavani-a-spristupnovani/.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice,* 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na http://doi.org/10.52036/1335793X.2023.3-4.10-22.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex--iso690/biblatex-iso690.pdf.

Termín odovzdania bakalárskej práce:	31. 05. 2025
Dátum schválenia zadania bakalárskej práce:	28. 02. 2025
Zadanie bakalárskej práce schválil:	tituly Meno Priezvisko, tituly – garant študijného programu

Poďakovanie

Týmto by som sa chcel úprimne poďakovať vedúcemu mojej diplomovej práce Ing. Marošovi Čavojskému, PhD. za jeho cenné rady, pomoc a vedenie počas písania tejto práce. Moje poďakovanie patrí taktiež mojej rodine za správnu výchovu, potrebunú k dopraqcovaniu až sem,mojej priateľke za psychickú podporu a poskytnutie príjemného prostredia pri písaní práce, ako aj Ministerstvu školstva za bezplatnú možnosť študovať na tejto fakulte.

Abstrakt

Táto diplomová práca sa zameriava na vývoj aplikácie, ktorá používateľom umožňuje jednoduché sociálne prepájanie prostredníctvom organizovania a vyhľadávania podujatí na základe ich polohy. Aplikácia podporuje vytváranie menších neformálnych stretnutí jednotlivcov so spoločnými záujmami, ako aj organizovanie väčších podujatí zo strany firiem a organizácií.

Cieľom práce bolo vytvoriť používateľsky prívetivú aplikáciu využívajúcu moderné postupy a technológie, ktorá zjednodušuje interakciu medzi používateľmi so spoločnými záujmami a podporuje živé stretnutia v reálnom čase.

Systém je navrhnutý ako Android aplikácia využívajúca GPS technológiu na zistenie polohy používateľa a poskytuje možnosti pre vytváranie a správu udalostí, pozývanie účastníkov a správu notifikácií. Aplikácia funguje aj na pozadí a je schopná posielať notifikácie. Využíva Firebase databázu pre ukladanie a synchronizáciu dát v reálnom čase, ako aj vlastný odporúčací systém pre navrhovanie udalostí na základe histórie používateľa a jeho záujmov. Aplikácia má taktiež implementovaný systém na detekciu nevhodných výrazov v názvoch a popisoch podujatí.

Výsledná aplikácia bola otestovaná na reálnych zariadeniach a hodnotená z hľadiska funkčnosti a použiteľnosti.

Kľúčové slová

záverečná práca, šablóna, LaTeX, formátovanie textu, citácie

Abstract

The manual for students of the Faculty of Electrical Engineering and Information Technology at Slovak University of Technology in Bratislava provides advice and guidelines on how to approach the formal aspects of writing a final thesis for university studies. The document can also serve as a template for the thesis in the LaTeX typesetting system. It covers in detail the rules for typesetting mathematical equations, numbering floating objects, and referencing them. It also thoroughly addresses the method of citing external literary sources.

Keywords

Final thesis, events, meeting, organizer, location, gps, kotlin, android

Obsah

Úvod **14**

1 Štruktúra záverečnej práce **15**

1.1 Základné pojmy 15
1.2 Architektúra a komunikácia v distribuovaných systémoch 15
1.2.1 Klient-server architektúra 15
1.2.2 Komunikačné rozhrania a protokoly (REST API a HTTP) . . . 15
1.2.3 Cloudové riešenia a služby 16
1.3 Správa a ukladanie dát 16
1.3.1 Relačné databázy (SQL) 17
1.3.2 Nerelačné databázy (NoSQL) 17
1.3.3 Real-time databázy 17
1.3.4 Autentifikácia 17
1.3.5 Faktory a metódy autentifikácie 17
1.3.6 Digitálna autentifikácia a bezpečnosť 18
1.3.7 Autorizácia 18
1.3.8 Mechanizmus trvalého prihlásenia (Token-based Authentication) 18
1.4 Vývoj mobilných aplikácií 19
1.4.1 Mobilná aplikácia 19
1.4.2 Programovacie jazyky 19
1.4.3 Softvérové frameworky 19
1.4.4 Vývojové prostredia a proces spracovania kódu 19
1.4.5 Prístupy k vývoju mobilných aplikácií 20
1.4.6 Technické obmedzenia a optimalizácia výkonu 20
1.4.7 Distribúcia a ekosystém platforiem 21
1.5 Geolokačné technológie a ich využitie 21
1.5.1 Operačná efektivita a hybridný prístup 22
1.5.2 Matematické metódy určovania polohy 22
1.6 Metódy verifikácie účasti na podujatí 22
1.7 Definícia a význam odporúčacích systémov 22
1.7.1 Obsahové filtrovanie (Content-Based Filtering) 23
1.7.2 Kolaboratívne filtrovanie (Collaborative Filtering) 23
1.7.3 Hybridné systémy 24
1.8 Techniky spracovania dát v odporúčacích systémoch 25
1.8.1 Dizajn a používateľská skúsenosť (UX/UI) 25
1.9 Architektonické vzory a princípy návrhu 26

1.10 Základné princípy moderného návrhu 26
1.11 Porovnanie architektonických vzorov 27

2 Súčasný stav riešenej problematiky 28
2.1 Facebook Events 28
2.2 Meetup 29
2.3 AllEvents.in 30
2.4 Hapsy 31
2.5 Mapibara 32
2.6 Zhrnutie analýzy súčasného stavu 33
2.7 Syntéza poznatkov a východiská pre vlastný návrh 34

3 Použité technológie 36
3.1 Vývojové prostredie a platforma 36
3.1.1 Operačný systém Android 36
3.1.2 Distribúcia a ekosystém platforiem 36
3.2 Programovací jazyk Kotlin 36
3.3 UI Toolkit: Jetpack Compose 37
3.4 Vývojové nástroje 37
3.5 Technická špecifikácia a použité knižnice 37
3.6 Android SDK a správa verzií 37
3.6.1 Dizajnový systém Material Design 3 38
3.7 Externé softvérové knižnice 39
3.7.1 Navigation Compose (2.8.5) 39
3.7.2 CameraX (1.3.4) 39
3.7.3 Coil (2.7.0) 40
3.8 Architektúra projektu 40
3.8.1 Moderný prístup: MVVM v kombinácii s Jetpack Compose . . . 41
3.9 Platforma Firebase 41
3.9.1 Dôvody voľby a kľúčové výhody 41
3.10 Firebase Authentication 42
3.10.1 Mechanizmus fungovania a správa tokenov 42
3.10.2 Prepojenie s Cloud Firestore a dokument users/{uid} 43
3.11 Firebase Firestore 44
3.12 Firebase Storage 44
3.13 Technológia QR kódov 44
3.13.1 Kľúčové vlastnosti a štruktúra 44
3.13.2 Využitie v mobilných zariadeniach 45

3.14 Cloudová infraštruktúra a Google Maps Platform 46
3.14.1 Globálna infraštruktúra: Regióny a Zóny 46
3.15 Cloudinary – správa digitálnych aktív 48
3.15.1 Kľúčové výhody a funkcie 48
3.15.2 Porovnanie s Firebase Storage a ekonomické hľadisko 48
3.16 Návrh a evolúcia algoritmu odporúčania 49

4 Návrh riešenia **50**
4.0.1 Funkcionálne a Nefunkcionálne požiadavky 50
4.0.2 Funkcionálne požiadavky 50
4.0.3 Nefunkcionálne požiadavky 52
4.0.4 UML diagramy 53
4.1 Implementacia 57
4.1.1 Autentifikácia a správa používateľov 57
4.2 Implementácia mapového modulu a geolokalizácie 59
4.3 Eventy — implementácia 60
4.3.1 Tvorba eventu 60
4.3.2 Detail eventu 62
4.3.3 Správa účasti 63
4.3.4 Filtrovanie udalostí 64
4.3.5 Hodnotenie udalostí 66
4.3.6 Check-in systém 68
4.3.7 Systém penalizácií (No-show) 70
4.3.8 Systém nahlasovania 73
4.3.9 Systém priateľstiev 75
4.4 Implementácia systému pozvánok 78
4.5 Implementácia systému lokálnych notifikácií 81

Záver **85**

Literatúra **87**
Použitie nástrojov umelej inteligencie 87

A Algoritmus **88**

B Výpis dlhého kódu **89**

C Slovníček pojmov **93**

Zoznam značiek a skratiek

$\vec{a}, \boldsymbol{a}$ – zrýchlenie (m/s^2)

a – veľkosť zrýchlenia (m/s^2), mriežková konštanta (nm)

AC – striedavý prúd, z angl. *Accelerating Current*

AFM – mikroskop atómových síl, z angl. *Atomic Force Microscope*

AI – umelá inteligencia, z angl. *Artificial Intelligence*

ε_0, ε_0 – elektrická konštanta ($\varepsilon_0 = 8{,}85418782\,.\,10^{-12}\,\mathrm{m^{-3}kg^{-1}s^4A^2}$)

GAI – generatívna umelá inteligencia

λ – vlnová dĺžka svetla (nm)

LED – svetlo emitujúca dióda, z angl. *Light-Emitting Diode*

LLM – veľký jazykový model, z angl. *Large Language Model*

$\vec{F}, \boldsymbol{F}, F$ – vektor sily, veľkosť sily (N)

q – elektrický náboj (C)

$\boldsymbol{r}$, r – polohový vektor, veľkosť polohového vektora (m)

SEM – skenovací elektrónový mikroskop

TEM – transmisný elektrónový mikroskop

U – elektrické napätie (V)

Zoznam algoritmov

1 Vypočítaj $y = x^n$ 88

Zoznam výpisov kódov

1 Registrácia používateľa v AuthService 57
2 Prihlásenie cez Google v AuthService 58
3 Prihlásenie pomocou e-mailu a hesla 58
4 Validácia hesla v RegisterViewModel 58
5 Inicializácia MapView v prostredí Compose 59
6 Získanie aktuálnej polohy používateľa 59
7 Konverzia modelu Event na MarkerOptions 59
8 Upload obrázka na Cloudinary 61
9 Uloženie eventu 61
10 Uloženie eventu do Firestore 62
11 Stavové premenné v detaile eventu 62
12 Prihlásenie na event 63
13 Odhlásenie z eventu 63
14 Odstránenie účastníka tvorcom eventu 63
15 Dátová trieda FilterCriteria 64
16 Implementácia EventFilterService 64
17 Haversine vzorec 65
18 Aplikovanie filtrov na mapu 66
19 Polia hodnotenia v dátovom modeli eventu 67
20 Implementácia metódy rateEvent 67
21 Podmienka zobrazenia sekcie hodnotenia 68
22 Generovanie QR kódu pomocou ZXing 68
23 Skenovanie QR kódu pomocou ML Kit 69
24 Logika check-inu v AppViewModel 70
25 Polia no-show v UserProfile 70
26 Výpočet počtu no-show za posledných 30 dní 71
27 Implementácia recordNoShow 71
28 Kontrola zablokovania pri prihlásení 72
29 Dátová trieda Report 73
30 Konštanty systému nahlasovania 73
31 Implementácia metódy reportEvent 74
32 Automatické zablokovanie používateľa 74
33 Dátová trieda Friendship 75
34 Určenie stavu priateľstva 75
35 Načítanie priateľstiev z Firestore 76

36 Odoslanie žiadosti o priateľstvo 76
37 Prijatie alebo odmietnutie žiadosti 77
38 Odstránenie priateľa 77
39 Deterministické generovanie ID konverzácie 78
40 Dátový model Invitation 78
41 Odoslanie pozvánky s kontrolou duplicity 79
42 Reakcia na pozvánku 80
43 Vytvorenie notifikačného kanála 81
44 Plánovanie notifikácie pomocou AlarmManager 82
45 Zostavenie a zobrazenie notifikácie 82
46 Zrušenie naplánovanej notifikácie 83
47 Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting` 89

Úvod

Digitalizácia priniesla veľa výhod: online nákupy, objednanie jedla jedným klikom, stretnutia cez internet bez fyzickej prítomnosti druhej osoby a mnoho ďalších vecí, ktoré nám uľahčujú život. Ale nájdu sa v tomto svete aj nevýhody?

Čím viac nás vtiahne tento virtuálny svet, tým viac zabúdame na to, čo je realita. Nakoniec zistíme, že nekonečné scrollovanie na sociálnych sieťach nám neprinesie do života dlhodobú radosť, skôr nám ho urobí nudnejším. Robí nás pasívnejšími a závislejšími od technológií. Talenty, ktoré sme možno kedysi mali, sa postupne vytrácajú. A aj keby sme mali chvíľkovú potrebu s tým niečo spraviť, zabrzdí nás fakt, že tomu musíme venovať úsilie a čas.

Ale čo ak by existovala jednoduchá aplikácia, kde si človek môže nájsť ľudí so spoločnými záujmami, akcie, ktoré sa mu páčia, alebo objaviť nové hobby len preto, lebo vyzerajú zaujímavo na fotkách? Toto je dôvod, prečo som sa rozhodol vytvoriť túto mobilnú aplikáciu.

Cieľom tejto diplomovej práce je vytvoriť mobilnú aplikáciu pre systém Android, ktorá spája ľudí, pomáha organizátorom jednoducho vytvárať vlastné podujatia a pomáha používateľom nájsť zábavu, nové záujmy alebo len tak tráviť čas s priateľmi. Pri tejto práci využijeme množstvo postupov a technológií, ktoré nám umožnia vytvoriť funkčnú a používateľsky príjemnú aplikáciu.

V práci sa budeme venovať návrhu a implementácii tejto aplikácie. Využijeme databázy na ukladanie informácií o používateľoch a podujatiach a navrhneme odporúčací systém vzhľadom na používateľovu históriu. Taktiež navrhneme systém, ako zamedziť zneužívaniu aplikácie a ako zabezpečiť kontrolu nevhodného obsahu. Aplikácia bude mať jednoduché a intuitívne rozhranie, ktoré umožní používateľom ľahko sa orientovať a využívať všetky funkcie. Nakoniec budeme aplikáciu testovať a získavať spätnú väzbu od používateľov, aby sme ju mohli vylepšiť a prispôsobiť ich potrebám. Verím, že táto aplikácia bude užitočná pre mnoho ľudí, pomôže im nájsť nové záujmy a že jej finálna verzia prinesie do tohto sveta hodnotu.

1 Štruktúra záverečnej práce

Táto diplomová práca je rozdelená na niekoľko logických celkov, ktoré postupne opisujú cestu od teoretickej analýzy až po finálnu implementáciu a testovanie aplikácie.

1.1 Základné pojmy

V tejto kapitole sa zameriame na všeobecné pojmy a technológie, ktoré sú kľúčové pre správne pochopenie problematiky vývoja mobilných aplikácií a implementácie odporúčacích systémov. Cieľom je poskytnúť čitateľovi pevný základ, na ktorom bude možné stavať v ďalších častiach práce.

1.2 Architektúra a komunikácia v distribuovaných systémoch

Základom modernej sieťovej komunikácie je efektívne rozdelenie úloh medzi jednotlivé uzly v sieti. Tento prístup umožňuje lepšiu správu zdrojov a vysokú škálovateľnosť aplikácií.

1.2.1 Klient-server architektúra

Architektúra klient-server predstavuje systém, ktorý vykonáva funkcie klienta aj servera s cieľom podporiť zdieľanie informácií medzi nimi. Tento model umožňuje viacerým používateľom pristupovať k rovnakej databáze v rovnakom čase, pričom databáza slúži na ukladanie veľkého množstva informácií [**sulyman2014**]. (Zdroj: `https://www.researchgate.net/profile/Shakirat-Sulyman/publication/271295146_Client-Server_Model/links/5864e11308ae8fce490c1b01/Client-Server-Model.pdf`)

- **Server:** Predstavuje počítač alebo softvérový systém, ktorý poskytuje dáta, zdroje alebo služby iným počítačom v sieti, ktoré označujeme ako klienti.
  (Zdroj: `https://en.wikipedia.org/wiki/Server_(computing)`)
- **Klient:** Ide o kus počítačového hardvéru alebo softvéru, ktorý pristupuje k službe sprístupnenej serverom v rámci modelu klient-server v počítačových sieťach.
  (Zdroj: `https://en.wikipedia.org/wiki/Client_(computing)`)

1.2.2 Komunikačné rozhrania a protokoly (REST API a HTTP)

Na to, aby si klient a server v distribuovanom systéme rozumeli, potrebujú spoločný komunikačný štýl a protokol. V súčasnosti je najrozšírenejším štandardom rozhranie Representational State Transfer (REST API). Ide o aplikačné programové rozhranie,

ktoré vyhovuje princípom architektonického štýlu REST. Jeho podstata spočíva v práci so zdrojmi, ktorými môže byť akákoľvek informácia, napríklad používateľ, produkt alebo dokument [**google_cloud_rest**].

Celý proces komunikácie prebieha v dvoch základných krokoch:

1. **Klient odošle požiadavku:** Mobilná aplikácia iniciuje požiadavku na vykonanie akcie smerom k API. Táto požiadavka obsahuje metódu HTTP, hlavičky s metadátami a prípadne telo s dátami.

2. **Server spracuje požiadavku:** Server prijme a overí požiadavku (autentifikácia a autorizácia). Následne ju spracuje, čo môže zahŕňať načítanie dát z databázy alebo vytvorenie nového zdroja.
   (Zdroj: `https://cloud.google.com/discover/what-is-rest-api`)

Samotným nosným médiom pre túto komunikáciu je protokol Hypertext Transfer Protocol (HTTP). Ide o základný aplikačný protokol, ktorý umožňuje komunikáciu a prenos dát v prostredí celosvetovej siete. V architektúre klient-server funguje ako „jazyk“, ktorým sa tieto dve strany dorozumievajú.
(Zdroj: `https://gemini.google.com/app/6ed6e44697b48104` – konzultácia 29.03.2026)

1.2.3 Cloudové riešenia a služby

Cloud môžeme prirovnať k virtuálnemu priestoru, prostredníctvom ktorého používatelia vytvárajú, zdieľajú a narábajú s obrovským množstvom informácií rôzneho typu. Ide o výpočtové zdroje dostupné na vyžiadanie (on-demand), ku ktorým má používateľ prístup cez internet [**eset_cloud**].

Dáta sa ukladajú na cloudové servery umiestnené v dátových centrách po celom svete a sú plne riadené ich poskytovateľom. Za poplatok môže zákazník na týchto serveroch prevádzkovať svoj softvér, využívať ich výpočtový výkon či úložisko na ukladanie vlastných dát.
(Zdroj: `https://bezpecnenanete.eset.com/sk/it-bezpecnost/co-je-to-cloud-a-ako-funguje/`)

1.3 Správa a ukladanie dát

Databáza funguje ako digitálne úložisko, ktoré organizuje dáta potrebné pre chod aplikácie. Podľa spôsobu ukladania a výkonu ich najčastejšie delíme na:

1.3.1 Relačné databázy (SQL)

Dáta sú v tomto modeli organizované do **tabuliek** s riadkami a stĺpcami. Medzi najpoužívanejšie systémy podľa portálu Webglobe [**webglobe_databaza**] patria:

- **MySQL:** Jednoduchá a rýchla databáza, hojne využívaná pri menších až stredne veľkých aplikáciách.
- **MariaDB:** Ponúka vyšší výkon a rozšírené funkcie, je vhodná aj pre náročnejšie aplikácie.
- **PostgreSQL:** Výkonná a robustná databáza s pokročilými funkciami, ideálna pre veľké a zložité aplikácie.

(Zdroj: `https://www.webglobe.sk/poradna/co-je-databaza`)

1.3.2 Nerelačné databázy (NoSQL)

Na rozdiel od tabuliek ukladajú dáta flexibilne, najčastejšie ako **dokumenty**. Tento typ (napr. Firebase Firestore) je kľúčový pre moderné mobilné aplikácie, pretože umožňuje meniť štruktúru dát „za pochodu“ bez nutnosti reštartovania systému [**aws_nosql**].
(Zdroj: `https://aws.amazon.com/nosql/`)

1.3.3 Real-time databázy

Špeciálny typ databázy, ktorý **okamžite synchronizuje zmeny** medzi serverom a všetkými pripojenými používateľmi bez nutnosti obnovovania aplikácie. Je to základ pre funkcie ako chaty alebo živé aktualizácie obsahu [**firebase_db**].
(Zdroj: `https://firebase.google.com/docs/database`)

1.3.4 Autentifikácia

Autentifikácia je proces overenia používateľa (overenia jeho identity), teda zistenie, o koho ide. Môže sa to overiť podľa kombinácie mena a hesla, biometrie (napr. otlačku prsta alebo rozpoznania hlasu), alebo iným zariadením.

https://sk.wikipedia.org/wiki/Autentifik

1.3.5 Faktory a metódy autentifikácie

V informatike delíme spôsoby overenia používateľa do troch základných kategórií podľa toho, na čom sú založené:

- **Znalostný faktor:** Niečo, čo používateľ pozná (napr. heslo, PIN).

- **Faktor vlastníctva:** Niečo, čo používateľ má (napr. smartfón, bezpečnostný token).
- **Dedičný faktor:** Niečo, čím používateľ je (biometria – odtlačok prsta, sken tváre).

Pre dosiahnutie vysokej bezpečnosti sa odporúča **viacfaktorové overenie** (MFA), ktoré kombinuje aspoň dva z vyššie uvedených faktorov. V mobilných zariadeniach sa čoraz častejšie využíva aj tzv. **kontinuálne overovanie**, ktoré neustále monitoruje identitu používateľa na základe biometrie správania (napr. štýl písania alebo dynamika dotyku).

1.3.6 Digitálna autentifikácia a bezpečnosť

Proces digitálneho overovania pozostáva z prihlásenia (vytvorenie poverení) a samotnej autentifikácie (predloženie dôkazu identity, napr. tokenu). Pri sieťovej komunikácii je kľúčové chrániť tento proces pred útokmi typu *Man-in-the-Middle* (MITM), kedy sa útočník snaží zachytiť prenášané údaje.
(Zdroj: `https://sk.wikipedia.org/wiki/AutentifikÃącia`)

1.3.7 Autorizácia

Autorizácia je proces overenia oprávnení pre danú úlohu, teda či autentifikovaný používateľ (systém) má oprávnenie vykonať úlohu, o ktorú žiada (napr. vstúpiť do systému alebo vykonať platobnú operáciu).

1.3.8 Mechanizmus trvalého prihlásenia (Token-based Authentication)

V moderných mobilných aplikáciách je žiaduce, aby používateľ nebol nútený zadávať prihlasovacie údaje pri každom otvorení aplikácie. Tento proces sa nazýva *perzistentná autentifikácia* a je realizovaný pomocou systému dvoch typov tokenov (bezpečnostných kľúčov):

- **Access Token (Prístupový kľúč):** Krátkodobý token (platnosť zvyčajne 60 minút), ktorý mobilná aplikácia pripája ku každej požiadavke na server. Slúži ako dôkaz, že používateľ je momentálne overený.
- **Refresh Token (Obnovovací kľúč):** Dlhodobý token uložený v šifrovanej pamäti zariadenia. Jeho jedinou úlohou je vyžiadať si nový *Access Token* v prípade, že pôvodný vypršal.

Tento cyklus prebieha na pozadí bez zásahu používateľa (tzv. *Silent Re-authentication*). Ak je *Refresh Token* stále platný, aplikácia automaticky obnoví reláciu, čím zabezpečí plynulý používateľský zážitok pri zachovaní vysokej bezpečnosti, keďže samotné heslo sa sieťou prenáša iba raz – pri prvotnom prihlásení.
(Zdroj: `https://auth0.com/docs/secure/tokens/refresh-tokens`)

1.4 Vývoj mobilných aplikácií

1.4.1 Mobilná aplikácia

Mobilná aplikácia je softvérová aplikácia vytvorená špecificky pre smartfóny, tablety alebo iné mobilné zariadenia, ktorá si vyžaduje inštaláciu, čím sa líši od bežného webu. V prvom rade vďaka nim dokážeme rozlíšiť smartfón od bežného telefónu.
(Zdroj: `https://www.eliteml.sk/blog/mobilne-aplikacie-v-skratke/`)

1.4.2 Programovacie jazyky

Programovací jazyk obsahuje inštrukcie vytvorené programátormi, ktoré riadia počítač pri vykonávaní konkrétnych úloh. Tieto inštrukcie, bežne vnímané ako komplexný kód, dodržiavajú odlišnú syntax špecifickú pre každý programovací jazyk a slúžia ako nevyhnutné nástroje pre vývoj softvéru a rôzne výpočtové úlohy.
(Zdroj: `https://codelabsacademy.com/sk/blog/what-is-a-programming-language`)

1.4.3 Softvérové frameworky

Framework je softvérová platforma alebo sada nástrojov, ktorá poskytuje štruktúru a základné funkcionality pre vývoj softvéru. Frameworky uľahčujú vývoj aplikácií tým, že poskytujú preddefinovanú štruktúru, architektúru a sadu nástrojov, ktoré zjednodušujú a urýchľujú proces vývoja.
(Zdroj: `https://coderama.com/slovnik/framework`)

1.4.4 Vývojové prostredia a proces spracovania kódu

Na písanie a správu programovacieho kódu využívajú vývojári špecializované softvérové nástroje, ktoré označujeme ako **IDE** (*Integrated Development Environment* – Integrované vývojové prostredie). Tieto nástroje siahajú od jednoduchých textových editorov s podporou syntaxe až po komplexné systémy, ktoré obsahujú:

- **Editor kódu:** Miesto na samotný zápis inštrukcií s funkciami ako automatické dopĺňanie kódu.

- **Debugger:** Nástroj na vyhľadávanie a opravu logických chýb v programe.
- **Kompilátor / Interpret:** Špeciálny program, ktorý transformuje (kompiluje) kód napísaný programátorom do binárnej podoby (strojového kódu), ktorej rozumie hardvér mobilného zariadenia alebo procesor.

Medzi najpoužívanejšie prostredia pre mobilný vývoj patria napríklad **Android Studio**, **Visual Studio Code** (populárne pre frameworky ako React Native alebo Flutter) či **Xcode** pre platformu iOS. Výber konkrétneho prostredia často závisí od zvoleného programovacieho jazyka a cieľovej platformy.
(Zdroj: `https://it-slovnik.cz/pojem/ide-integrovane-vyvojove-prostredi`)

1.4.5 Prístupy k vývoju mobilných aplikácií

Podľa spoločnosti IBM možno proces vývoja mobilných aplikácií definovať ako tvorbu softvéru navrhnutého špecificky pre mobilné koncové zariadenia, pričom kľúčovým rozhodnutím je technológia implementácie. V súčasnosti sa vývojári rozhodujú primárne medzi natívnym a multiplatformovým prístupom [**pullur_medium_2025**]:

- **Natívne aplikácie:** Sú vyvíjané priamo pre konkrétnu platformu (Android, iOS) pomocou dedikovaných nástrojov. Ponúkajú najvyšší výkon a neobmedzený prístup k hardvéru.
- **Webové mobilné aplikácie:** Ide o responzívne webové stránky využívajúce technológie HTML5, CSS a JavaScript. Modernou formou sú tzv. *Progressive Web Apps* (PWA).
- **Hybridné aplikácie:** Kombinujú predchádzajúce prístupy. Jadro aplikácie je písané vo webových technológiách, ktoré sú následne "zabalené"do natívneho kontajnera.

(Zdroj: `https://opac.crzp.sk/?fn=detailBiblioFormChildO1BU&sid=9A7D98113A4E7D6F09AF71A0EB46&seo=CRZP-detail-kniha`)

1.4.6 Technické obmedzenia a optimalizácia výkonu

Vývoj pre mobilné platformy vyžaduje špecifický prístup k správe prostriedkov, ktoré sú v porovnaní s desktopovými systémami limitované:

1. **Energetická a systémová efektivita:** Natívne aplikácie vykazujú vyššiu energetickú efektivitu, čo je kľúčové pri aplikáciách využívajúcich lokalizačné služby (GPS) [**pullur_medium_2025**].

2. **Cloudové spracovanie (*Cloud Offloading*):** Náročné výpočtové operácie, ako sú komplexné odporúčacie algoritmy, je vhodné delegovať na cloudové servery (napr. Firebase).

3. **Odozva používateľského rozhrania (UI):** Natívne frameworky dosahujú najlepšiu plynulosť animácií a rýchlosť odozvy na dotyk.

1.4.7 Distribúcia a ekosystém platforiem

Platforma Android, s približne 70 % trhovým podielom, je často preferovaná pre nasadenie MVP kvôli flexibilnejším pravidlám. Naopak, ekosystém iOS vyžaduje striktnejšie dodržiavanie štandardov. Multiplatformové nástroje ako Flutter a React Native umožňujú súčasné nasadenie na oboch platformách s určitým kompromisom v oblasti výkonu. (Zdroj: `https://medium.com/@riyaspullurofficial/native-vs-cross-platform-app-size-and-performance-comparison-550cb7f918b0`)

Tabuľka 1: Súhrnné porovnanie vývojových prístupov (podľa [**pullur_medium_2025**])

Metrika	**Native**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka

1.5 Geolokačné technológie a ich využitie

Geolokácia je proces identifikácie geografickej polohy zariadenia. Podľa analýzy spoločnosti Sensolus [**sensolus_geoloc**] sa v moderných systémoch využívajú štyri hlavné technológie, ktoré sa líšia presnosťou a energetickou náročnosťou:

- **GPS (Global Positioning System):** Využíva satelitné signály. Je najpresnejší v exteriéri (na metre), ale má vysokú spotrebu batérie a nefunguje vo vnútri budov.

- **Wi-Fi Sniffing:** Určuje polohu skenovaním okolitých Wi-Fi sietí. Je ideálny pre mestské prostredie a prechod medzi budovami, pričom spotrebúva menej energie ako GPS.

- **BLE (Bluetooth Low Energy):** Využíva malé vysielače (beacons). Poskytuje extrémnu presnosť v interiéroch (napr. sklady), ale vyžaduje inštaláciu fyzických zariadení v priestore.

- **LPWAN Geolocation:** Využíva nízkoenergetické siete (napr. Sigfox) na trianguláciu signálu. Má najdlhšiu výdrž batérie (roky), ale nižšiu presnosť (stovky metrov).

1.5.1 Operačná efektivita a hybridný prístup

Výber správnej technológie priamo ovplyvňuje prevádzkovú efektivitu systému. V mobilnom vývoji sa najčastejšie využíva tzv. **hybridná geolokácia**, ktorá inteligentne prepína medzi GPS a Wi-Fi podľa dostupnosti signálu a stavu batérie. To umožňuje aplikáciám efektívne sledovať polohu používateľa bez nadmerného zaťaženia hardvéru. (Zdroj: `https://www.sensolus.com/knowledge-hub/behind-the-tech/four-geolocation-technologies-compared-how-can-they-improve-your-operational-efficiency/`)

1.5.2 Matematické metódy určovania polohy

Na výpočet presnej polohy zariadenia zo získaných signálov (rádiových, satelitných či Bluetooth) sa v informatike využívajú nasledujúce geometrické a fyzikálne metódy:

- **TOA (Time of Arrival):** Metóda meria absolútny čas, za ktorý signál prejde od vysielača k prijímaču. Na určenie polohy v 2D priestore sú potrebné aspoň tri vysielače (trilaterácia).
- **TDOA (Time Difference of Arrival):** Nameria sa rozdiel v čase príchodu signálov z viacerých vysielačov. Je to praktickejšie ako TOA, pretože nevyžaduje dokonalú synchronizáciu hodín medzi vysielačom a mobilným zariadením.
- **AOA (Angle of Arrival):** Táto metóda neurčuje vzdialenosť, ale **uhol**, pod ktorým signál prichádza k anténe. Kombináciou uhlov z dvoch staníc je možné určiť presný bod polohy (triangulácia).
- **RSS (Received Signal Strength):** Odhaduje vzdialenosť na základe útlmu sily signálu. Čím je signál slabší, tým je zariadenie pravdepodobne ďalej od vysielača (často využívané pri Wi-Fi a Bluetooth).

Tieto metódy sú matematickým základom pre fungovanie GPS (prevažne TOA/TDOA) aj moderných systémov pre lokalizáciu v interiéroch.

1.6 Metódy verifikácie účasti na podujatí

1.7 Definícia a význam odporúčacích systémov

S prudkým rozvojom internetových technológií a inteligentných mobilných zariadení čelí moderný používateľ fenoménu informačného preťaženia. V prostredí s takmer

neobmedzeným množstvom digitálneho obsahu a služieb sa odporúčacie systémy (angl. *Recommender Systems*) stali kľúčovým nástrojom na personalizovanú filtráciu informácií.

Ich primárnou úlohou je predpovedať preferencie používateľa a zredukovať rozsiahly priestor dostupných položiek na úzky výber relevantných možností, čím sa zvyšuje celková spokojnosť a efektivita interakcie s platformou. Súčasný výskum sa čoraz viac orientuje na spracovanie implicitných dát. Ide o behaviorálne vzorce, ako sú kliknutia (*click-stream*), história navštívených miest či čas strávený interakciou s konkrétnym objektom [**electronics11010141**].

1.7.1 Obsahové filtrovanie (Content-Based Filtering)

Tento prístup je založený na analýze špecifických atribútov položiek. Systém identifikuje vlastnosti objektov (napr. textové informácie, kategórie alebo tagy), o ktoré používateľ v minulosti prejavil záujem, a na ich základe generuje odporúčania pre podobné položky [**electronics11010141**].

- **Princíp:** Analýza atribútov a metadát položiek (napr. žáner, kategória) s cieľom nájsť podobnosť s preferenciami používateľa.
- **Využitie:** Efektívne najmä pri službách pracujúcich s textovými dátami, hudbou alebo e-commerce [**electronics11010141**].
- **Limity:** Nízka miera diverzity (angl. *serendipity*) – používateľovi nie sú ponúkané nové, neobjavené kategórie obsahu, čo vedie k efektu informačnej bubliny.

1.7.2 Kolaboratívne filtrovanie (Collaborative Filtering)

Kolaboratívne filtrovanie predpovedá vkus používateľa na základe analýzy správania a hodnotení celej komunity používateľov (tzv. „wisdom of the crowd“).

- **Memory-based:** Hľadá podobnosť priamo medzi používateľmi (*User-based*) alebo položkami (*Item-based*) pomocou algoritmov ako KNN alebo korelačných koeficientov.
- **Model-based:** Využíva pokročilé štatistické metódy a techniky strojového učenia, ako sú zhlukovanie (*Clustering*), *Singular Value Decomposition* (SVD) alebo *Principal Component Analysis* (PCA).
- **Hlavné nedostatky:** Zdroj [**electronics11010141**] definuje tri kritické obmedzenia:
  - *Cold Start:* Problém s odporúčaním položiek pre nových používateľov bez histórie interakcií.

- *Sparsity (Riedkosť dát):* Situácia, keď je v systéme príliš málo hodnotení na to, aby bolo možné spoľahlivo vypočítať podobnosť.
- *Gray Sheep (Sivé ovce):* Používatelia s veľmi unikátnym vkusom, ktorí nevykazujú podobnosť so žiadnou väčšou skupinou používateľov, čo sťažuje generovanie odporúčaní.

1.7.3 Hybridné systémy

Hybridné modely kombinujú viaceré techniky s cieľom eliminovať slabiny jednotlivých prístupov, najmä problém riedkosti dát (*sparsity*) a studeného štartu. Podľa Burkeho klasifikácie [**electronics11010141**] existuje sedem základných metód hybridizácie, ktoré sú uvedené v tabuľke 2. Hlavným cieľom väčšiny štúdií zaoberajúcich sa hybridnými

Tabuľka 2: Typy hybridných odporúčacích systémov podľa Burkeho [**electronics11010141**]

Metóda	**Princíp fungovania**
Weighted	Skóre z rôznych techník sa kombinuje pomocou numerických váh.
Switching	Systém prepína medzi technikami v závislosti od kontextu.
Mixed	Odporúčania z viacerých zdrojov sú zobrazené súčasne.
Feature Combination	Atribúty z jedného zdroja slúžia ako vstupné črty pre druhý model.
Feature Augmentation	Výstup jednej techniky sa stáva parametrom pre ďalšie spracovanie.
Cascade	Postupné zjemňovanie poradia (jeden systém vytvorí rebríček, druhý ho upresní).
Meta-level	Celý model vygenerovaný jednou technikou slúži ako vstup pre druhú.

modelmi po roku 2010 je integrácia tzv. vedľajších informácií (*side information*), ako sú sociálne väzby medzi používateľmi alebo kontextové dáta, ktoré dopĺňajú chýbajúce explicitné hodnotenia. Týmto spôsobom hybridné systémy dosahujú vyššiu presnosť a robustnosť v porovnaní so samostatnými modelmi.

Kritérium	Collaborative	Content	Hybrid	Knowledge
Cold Start (používateľ)	Zlé	Zlé	Stredné	Dobré
Cold Start (položka)	Zlé	Dobré	Stredné	Dobré
Škálovateľnosť	Zlé	Dobré	Stredné	Dobré
Diverzita	Dobré	Zlé	Dobré	Stredné
Potreba metadát	Nie	Áno	Čiastočne	Áno
Vysvetliteľnosť	Stredné	Dobré	Stredné	Dobré

Tabuľka 3: Porovnanie typov odporúčacích systémov

Porovnanie systémov

1.8 Techniky spracovania dát v odporúčacích systémoch

Pre efektívne fungovanie vyššie spomínaných modelov sa využívajú špecifické techniky dolovania dát (Data Mining):

1. **Text Mining:** Analýza neštruktúrovaných textov pomocou sémantickej analýzy a váhovania pojmov (napr. TF-IDF). Slúži na pochopenie obsahu podujatí a preferencií používateľa.

2. **K-Nearest Neighbor (KNN):** Algoritmus slúžiaci na klasifikáciu položiek alebo používateľov na základe ich blízkosti v metrickom priestore (geografickom alebo vkusovom).

3. **Clustering (Zhlukovanie):** Automatické rozdeľovanie dát do skupín so spoločnými črtami. Využíva sa najmä pri segmentácii používateľov alebo kategorizácii lokalít.

4. **Matrix Factorization:** Pokročilá technika rozkladu matíc, ktorá transformuje interakcie používateľov do priestoru latentných faktorov, čím rieši problém riedkosti dát (Sparsity).

5. **Neural Networks:** Modely hlbokého učenia schopné modelovať nelineárne vzťahy a spracovávať komplexné dáta zo sociálnych sietí či senzorov mobilných zariadení.

1.8.1 Dizajn a používateľská skúsenosť (UX/UI)

Pri vývoji mobilnej aplikácie nie je kľúčová len funkčná stránka (backend), ale aj spôsob, akým používateľ s aplikáciou interaguje. Návrh používateľského rozhrania sa riadi dvoma základnými disciplínami:

- **UX (User Experience):** Zameriava sa na celkový pocit používateľa pri používaní aplikácie. Cieľom je, aby bola aplikácia intuitívna, logicky usporiadaná a riešila potreby používateľa s minimálnym úsilím.
- **UI (User Interface):** Rieši vizuálnu stránku aplikácie – farby, typografiu, ikony a rozloženie prvkov na obrazovke.

1.9 Architektonické vzory a princípy návrhu

Úspešná mobilná aplikácia musí byť postavená na robustnej architektúre, ktorá zaručuje jej škálovateľnosť a udržateľnosť v ekosystéme rôznych zariadení (telefóny, tablety, ChromeOS či skladacie zariadenia). Podľa oficiálnej dokumentácie spoločnosti Google je základným cieľom architektúry definovať hranice medzi jednotlivými časťami aplikácie a jasne určiť ich zodpovednosť.

1.10 Základné princípy moderného návrhu

Pred definovaním konkrétnych vzorov je potrebné určiť pravidlá, ktoré musí moderná architektúra spĺňať pre zachovanie stability v prostredí s obmedzenými systémovými prostriedkami:

- **Separation of concerns (Oddelenie zodpovedností):** Kľúčový princíp, ktorý zakazuje sústredenie kódu v triedach *Activity* alebo *Fragment*. Tieto komponenty slúžia primárne ako hostitelia používateľského rozhrania, sú krátkodobé (*ephemeral*) a operačný systém ich môže kedykoľvek zničiť z dôvodu nedostatku pamäte alebo zmeny konfigurácie.
- **Drive UI from data models (Riadenie UI dátovými modelmi):** Používateľské rozhranie by malo byť postavené na dátových modeloch, ktoré sú nezávislé od životného cyklu UI prvkov. Perzistentné modely zabezpečujú, že aplikácia nestratí dáta pri reštarte procesu a zostáva funkčná aj pri nestabilnom pripojení k sieti.
- **Single Source of Truth (SSOT - Jednotný zdroj pravdy):** Každý dátový typ má priradeného svojho vlastníka, ktorý ako jediný môže dáta modifikovať. Zmeny sa následne propagujú prostredníctvom imutabilných typov, čo znižuje výskyt chýb a uľahčuje ladenie systému.
- **Unidirectional Data Flow (UDF - Jednosmerný tok dát):** Stav aplikácie prúdi smerom nadol (od SSOT k UI elementom) a udalosti/akcie prúdia smerom nahor. Tento vzor zaručuje konzistenciu dát v celom systéme.

1.11 Porovnanie architektonických vzorov

V histórii vývoja platformy Android sa vyvinulo niekoľko prístupov k organizácii kódu, ktoré reagovali na narastajúcu komplexnosť aplikácií:

1. **MVC (Model-View-Controller):** Tradičný model, kde Controller riadi tok dát. V Androide tento vzor zlyhával, pretože triedy *Activity* preberali úlohu controllera aj view súčasne, čo viedlo k vzniku ťažko udržiavateľného kódu.

2. **MVP (Model-View-Presenter):** Oddelil zodpovednosť medzi View a Presenter. Problémom zostala vysoká previazanosť a nutnosť manuálneho ošetrovania životného cyklu, čo spôsobovalo úniky pamäte (*memory leaks*).

3. **MVVM (Model-View-ViewModel):** Súčasný štandard. *ViewModel* uchováva stav aplikácie nezávisle od UI komponentov, čím rieši problém s reštartovaním aktivít a stratu dát pri zmene orientácie zariadenia.

2 Súčasný stav riešenej problematiky

Na trhu existuje množstvo platforiem zameraných na organizáciu a objavovanie podujatí. Dominantným hráčom v tejto oblasti je služba **Facebook Events**, vyvíjaná spoločnosťou Meta. Vďaka zázemiu veľkej technologickej spoločnosti ide o komplexný systém, ktorý pokrýva široké spektrum potrieb používateľov.

2.1 Facebook Events

Facebook Events umožňuje používateľom vytvárať vyhradené stránky pre konkrétne príležitosti a zasielať verejné alebo súkromné pozvánky. Pre online firmy predstavujú tieto podujatia efektívny nástroj na propagáciu predajov, osláv alebo dôležitých míľnikov. Systém zahŕňa históriu podujatí, možnosť označovania obľúbených aktivít a pokročilé filtrovanie.

- **Výhody:** Obrovská používateľská základňa, vysoká miera integrácie so sociálnou sieťou, prepracované algoritmy na predikciu záujmov a množstvo funkcií dostupných bez priamych poplatkov za vytvorenie podujatia.
- **Nevýhody a obmedzenia:**
  - **Informačné zahltenie a notifikačná únava:** Rozhranie je preplnené reklamami. Nadmerné množstvo systémových upozornení (*Notification Fatigue*) často vedie k tomu, že používatelia dôležité pozvánky na podujatia prehliadnu alebo ignorujú.
  - **Nízka miera kontroly nad dosahom:** Organizátor nemá záruku, že sa informácia o podujatí dostane ku všetkým sledovateľom. Algoritmus uprednostňuje platenú reklamu a interaktívny obsah, čo znevýhodňuje menšie lokálne a komunitné aktivity.
  - **Uzavretosť ekosystému:** Plné využitie funkcií vyžaduje existenciu Facebook profilu, čo odrádza časť modernej populácie (najmä generáciu Z).
  - **Obmedzená vizualizácia:** Mapa podujatí nie je primárnym ovládacím prvkom a je dostupná až po manuálnom rozkliknutí, čo znižuje intuitívnosť pri hľadaní aktivít v okolí.
  - **Absencia priamej komunikácie:** Diskusia v rámci podujatia prebieha formou príspevkov na nástenke, chýba však funkcia reálneho skupinového chatu pre účastníkov v reálnom čase.

Zdroje:
`https://www.facebook.com/events/`
`https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother` https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother
Zdroje:
`https://www.facebook.com/events/`
`https://www.bigcommerce.com/glossary/what-are-facebook-events/`

2.2 Meetup

Platforma **Meetup** je celosvetovo známa platforma sa primárne zameriava na budovanie dlhodobých komunít a záujmových skupín (*Groups*). Na rozdiel od Facebooku, kde sú podujatia často jednorazové, Meetup motivuje používateľov k pravidelnému stretávaniu sa na základe spoločných koníčkov, ako sú technológie, šport či jazykové vzdelávanie.

V porovnaní s Facebook Events ponúka Meetup detailnejšie komunitné funkcie, avšak väčšina z nich je podmienená plateným členstvom:

- **Rozšírená interakcia:** Platforma umožňuje prehliadať zoznamy účastníkov, pristupovať k ich profilom a využívať pokročilé systémy komentárov a správ, čo uľahčuje nadväzovanie kontaktov ešte pred samotným podujatím.
- **Personalizácia (*For You*):** Algoritmus navrhuje podujatia a skupiny na základe histórie aktivít a preferencií používateľa, pričom prioritizuje lokálne komunity.
- **Platený model (Monetizácia):** Najväčšou nevýhodou platformy je, že kľúčové funkcie pre organizátorov (vytváranie podujatí, správa členov) aj pokročilé funkcie pre účastníkov sú spoplatnené. V rámci prémiového balíka získavajú používatelia:
  - prístup k úplným zoznamom a profilom účastníkov,
  - prioritný prístup k podujatiam a čakacím listinám (*Waitlists*),
  - neobmedzené zasielanie správ ostatným členom,
  - rozhranie bez reklám a včasné upozornenia na nové udalosti.
- **Problém s účasťou:** Podobne ako pri iných bezplatných registráciách, aj Meetup čelí vysokej miere tzv. *no-shows* (používatelia, ktorí potvrdia účasť, ale neprirodzene absentujú), čo sťažuje plánovanie kapacity pre organizátorov.

Kritické zhodnotenie: Hoci Meetup poskytuje kvalitnejšie nástroje na sieťovanie (*networking*) než Facebook, jeho uzavretosť za platobnú bránu a slabšia SEO viditeľnosť pre nečlenov otvárajú priestor pre vývoj prístupnejších, bezplatných alternatív.

- **Problém „No-shows“ (Neprihlásení účastníci):** Štatistiky z praxe ukazujú, že v priemere až 50 % používateľov, ktorí potvrdia účasť (*RSVP*), na udalosť nakoniec nepríde. Tento jav je spôsobený nízkou bariérou záväzku, najmä pri bezplatných podujatiach.

- **Vysoké prevádzkové náklady:** Pre organizátorov predstavuje Meetup finančnú záťaž (poplatky od cca 25 $ mesačne), pričom základné balíky výrazne obmedzujú možnosti brandingu, vkladanie multimédií (video, vlastné logá) či prispôsobenie dizajnu skupiny.

- **Geografické obmedzenia vyhľadávania:** Systém je primárne postavený na vyhľadávaní podľa PSČ alebo mesta. To drasticky limituje *SEO* dosah udalostí, ktoré sú organizované online alebo cília na globálne publikum, nakoľko používatelia nevidia relevantné výsledky mimo svojho bezprostredného okolia.

- **Pasivita a „Ghost groups“:** Na platforme sa nachádza veľké množstvo neaktívnych skupín, ktoré slúžia len na zber e-mailových adries (*lead generation*) bez reálnej snahy organizovať stretnutia.

- **Kvalita a bezpečnosť lokalít:** Mnohé stretnutia sa odohrávajú v baroch alebo reštauráciách, čo prináša problémy s hlukom, nevhodným osvetlením alebo dokonca neprofesionálnym správaním účastníkov (alkohol). Verejné inštitúcie (knižnice) sú často pre neformálne skupiny nedostupné.

- **Chýbajúca automatizácia časových pásiem:** Meetup nedostatočne rieši automatický prepočet časových pásiem pri medzinárodných online podujatiach, čo núti organizátorov k manuálnym opravám.

Zhrnutie pre návrh novej aplikácie:
Tieto zistenia naznačujú, že moderná alternatíva by sa mala zamerať na: 1. **Zvýšenie záväznosti účastníkov** (napr. cez integrovaný chat alebo potvrdzovací systém krátko pred akciou). 2. **Transparentnú mapu bez hraníc miest**, ktorá umožní objavovať udalosti organicky scrollovaním. 3. **Natívnu podporu pre online podujatia** s automatickou prácou s časom a SEO optimalizáciou.

2.3 AllEvents.in

AllEvents sa prezentuje ako globálna platforma na objavovanie udalostí, ktorá pokrýva viac než 40 000 miest po celom svete. Platforma sa zameriava na agregáciu širokého spektra podujatí – od komunitných stretnutí až po veľké komerčné koncerty a festivaly.

- **Deklarované funkcie:** Personalizované odporúčania na základe záujmov, sledovanie aktivít priateľov, bezpapierová registrácia (*check-in*) a priama rezervácia vstupeniek v rámci aplikácie.
- **Kategorizácia a distribúcia:** Platforma vyniká prehľadným členením do kategórií, možnosťou exportu do kalendára a funkciou priameho kontaktovania organizátora (*Message to host*).
- **Identifikované nedostatky a technické bariéry:**
  - **Kritická nestabilita aplikácie:** Pri testovaní mobilnej aplikácie boli zaznamenané závažné chyby v procese autentifikácie (*Login failed*) a zlyhania pri verifikácii používateľa cez mobilné číslo, čo znemožňuje plnohodnotné využívanie služieb.
  - **Lokalizačné problémy:** Napriek deklarovanému globálnemu dosahu systém v niektorých prípadoch nedokáže korektne identifikovať menšie mestá, čo núti používateľa k prehliadaniu v režime hosťa (*Guest mode*).
  - **Absencia sociálnej interakcie:** Hoci platforma umožňuje nákup lístkov a sledovanie udalostí, postráda funkciu reálneho skupinového chatu medzi účastníkmi a transparentné zobrazenie zoznamu prihlásených osôb.
  - **Zameranie obsahu:** Databáza je orientovaná primárne na veľké komerčné a spoločenské akcie, čím opomína drobné, spontánne komunitné aktivity.

Zdroje:
`https://play.google.com/store/apps/details?id=com.amitech.allevents`
`https://allevents.in/`

2.4 Hapsy

Aplikácia *Hapsy* sa profiluje ako agregátor podujatí, ktorého cieľom je zjednotiť ponuku z rôznych platforiem (Facebook, Eventbrite, Meetup, GoOut) do jedného interaktívneho zobrazenia na mape. Jej hlavnou ambíciou je eliminovať potrebu prepínania medzi viacerými aplikáciami.

- **Kľúčové funkcie:** Agregácia dát z externých zdrojov, personalizované upozornenia na základe preferencií používateľa, ukladanie obľúbených udalostí a prehľadná mapa mesta.
- **Identifikované obmedzenia a používateľská kritika:**

- **Geografická limitácia:** Podľa spätnej väzby používateľov trpí aplikácia nedostatkom lokálneho obsahu mimo hlavných miest. Algoritmus sa zameriava na veľké komerčné databázy, čím opomína komunitný život v menších regiónoch.
- **Absencia tvorby obsahu:** Systém funguje primárne ako prehliadač (*viewer*). Bežný používateľ nemá možnosť vytvoriť vlastnú udalosť, čo znemožňuje organizáciu spontánnych aktivít (napr. spomínané amatérske športové zápasy).
- **Technická nedokonalosť:** Používateľské recenzie naznačujú prítomnosť kritických chýb v rozhraní (*UI bugs*), ako sú problémy s posúvaním obsahu (*scrolling*), čo znižuje komfort používania.
- **Obmedzená interakcia:** Podobne ako pri iných agregátoroch, aj tu chýba sociálna vrstva – možnosť priamej komunikácie medzi účastníkmi alebo správa tímov.

Zdroje:
`https://play.google.com/store/apps/details?id=com.hapsy.app`
`https://mapibara.com/`

2.5 Mapibara

Platforma *Mapibara* predstavuje minimalistický prístup k objavovaniu podujatí, pričom kladie absolútny dôraz na mapové rozhranie a anonymitu používateľa. Na rozdiel od komplexných sociálnych sietí sa zbavuje povinných registrácií a zameriava sa na okamžitý prístup k informáciám.

- **Silné stránky dizajnu:** Extrémne jednoduché a prehľadné používateľské rozhranie (*UI*), kde mapa zaberá celú plochu obrazovky. Absencia prihlasovacieho procesu odstraňuje vstupnú bariéru pre nových používateľov.
- **Identifikované kritické nedostatky:**
  - **Absencia obsahu:** Pri praktickom testovaní platforma vykazuje takmer nulovú naplnenosť podujatiami, čo ju robí v súčasnom stave nepoužiteľnou.
  - **Chýbajúca interakcia:** Kvôli absencii používateľských profilov nie je možné s účastníkmi komunikovať, spravovať kapacity ani riešiť logistiku (napr. spomínané rozdelenie do tímov pri športe).
  - **Pasivita systému:** Platforma slúži len ako statický prehliadač bodov na mape bez možnosti aktívneho zapojenia komunity.

Ponaučenie pre vlastný návrh: Analýza Mapibary ukazuje, že samotná vizualizácia na mape nestačí. Pre úspech aplikácie je nevyhnutné skombinovať **jednoduchosť mapy** so **sociálnou interakciou** (prihlásenie, chat), aby používatelia mali dôvod podujatia nielen hľadať, ale aj vytvárať a navštevovať.

2.6 Zhrnutie analýzy súčasného stavu

Na základe podrobnej analýzy dominantných platforiem (Facebook Events, Meetup, AllEvents, Hapsy a Mapibara) je možné definovať súbor štandardných funkcií, ktoré tvoria základ moderných systémov na správu podujatí, ale zároveň identifikovať kritické funkčné medzery.

Štandardné funkcie prítomné v analyzovaných riešeniach: Väčšina spomenutých aplikácií disponuje nasledujúcim funkčným základom:

- **Objavovanie a vizualizácia:** Interaktívna mapa s podujatiami, vyhľadávanie podľa kategórií a pokročilé filtrovanie.
- **Informačná vrstva:** Detailný popis podujatia, fotogaléria, profil organizátora a zdieľanie (*sharing*) na iné platformy.
- **Personalizácia:** História navštívených udalostí, systém odporúčaných podujatí (*Recommended for you*) a push notifikácie.
- **Interakcia:** Možnosť vytvorenia vlastnej verejnej alebo súkromnej udalosti po predchádzajúcej registrácii.

Identifikácia funkčných medzier (Inovačný potenciál): Napriek širokej škále funkcií analýza odhalila oblasti, ktoré súčasné riešenia pokrývajú nedostatočne, čo vytvára priestor pre vývoj nového, špecializovaného riešenia:

- **Dynamický manažment kapacity:** Absencia automatizovaného systému náhradníkov (*Waiting List*). Väčšina platforiem neponúka riešenie pre prípady, kedy je kapacita naplnená a nie je možné automaticky osloviť ďalšieho záujemcu v poradí pri uvoľnení miesta.
- **Verifikácia reálnej účasti:** Kritický nedostatok nástrojov na potláčanie *no-show* efektu (používatelia, ktorí potvrdia účasť, ale neprídu). Chýbajú mechanizmy ako **GPS Check-in** alebo verifikácia prítomnosti na mieste činu.
- **Komunitná logistika:** Absentujú nástroje na správu spoločného vybavenia (*Inventory sharing* – napr. kto prinesie športové náčinie) a funkcia reálneho skupinového chatu pre všetkých účastníkov v reálnom čase.

- **Moderovanie a bezpečnosť:** Pri raste používateľmi generovaného obsahu chýbajú v mnohých aplikáciách citlivé nástroje na nahlasovanie (*reporting*) a včasné blokovanie nevhodného obsahu alebo používateľov.

- **Regionálna relevantnosť:** Nízka hustota obsahu v menších mestách a regiónoch mimo hlavných metropol, čo je dôsledkom orientácie veľkých platforiem na komerčné a masové podujatia.

2.7 Syntéza poznatkov a východiská pre vlastný návrh

Na základe vykonanej analýzy existujúcich riešení je možné konštatovať, že hoci trh ponúka robustné platformy, žiadna z nich nepokrýva komplexne potreby malých, spontánnych komunít s dôrazom na logistiku a potláčanie negativity (napr. spomínaný *no-show* efekt).

Pre lepšiu prehľadnosť sú kľúčové rozdiely medzi analyzovanými platformami a navrhovaným riešením zhrnuté v Tabuľke 4.

Tabuľka 4: Porovnanie kľúčových funkcií existujúcich platforiem a navrhovanej aplikácie

Funkcia / Platforma	**FB**	**Meetup**	**AllEvents**	**Hapsy**	**Mapibara**
Mapa ako primárne UI	Nie	Čiastočne	Áno	Áno	Áno
Tvorba udalostí	Áno	Platené	Áno	Nie	Nie
Skupinový chat	Nie	Áno	Nie	Nie	Nie
Manažment náhradníkov	Nie	Áno	Nie	Nie	Nie
Ochrana „No-show“	Nie	Nie	Nie	Nie	Nie
Zameranie na komunity	Áno	Áno	Nie	Nie	Nie

Definovanie smerovania vlastného riešenia Zistené nedostatky konkurenčných riešení priamo formujú funkčné požiadavky na moju aplikáciu. Vlastné riešenie sa bude odlišovať predovšetkým v troch pilieroch:

1. **Interaktivita namiesto pasivity:** Integrácia komunikačného modulu (chat) priamo do detailu udalosti, čo umožní operatívne riešiť zmeny (napr. počasie, meškanie) v reálnom čase.

2. **Zvýšenie kredibility účastníkov:** Implementácia overovacích mechanizmov (napr. potvrdzovanie prítomnosti na mieste cez polohu zariadenia), čo motivuje používateľov k reálnej účasti.

3. **Minimalizmus a dostupnosť:** Odstránenie finančných bariér pre organizátorov a zachovanie čistého mapového rozhrania pre okamžitý prehľad o dianí v okolí bez zbytočného informačného balastu.

Tento prístup umožní vytvoriť nástroj, ktorý nie je len „nástenkou podujatí“, ale živým ekosystémom pre lokálne komunity.
Zhrnutím možno konštatovať, že existuje dopyt po riešení, ktoré by premostilo priepasť medzi statickou informáciou o udalosti a aktívnou logistickou podporou komunitných aktivít.

3 Použité technológie

3.1 Vývojové prostredie a platforma

3.1.1 Operačný systém Android

Táto práca je zameraná na vývoj mobilnej aplikácie pre operačný systém **Android**. Dôvodom zvolenia tejto platformy je jej dominantné postavenie na trhu mobilných zariadení, kde podľa aktuálnych štatistík zaujíma viac ako 70 % podiel. Android ponúka široké možnosti prispôsobenia a hlbokej integrácie s hardvérovými službami (ako sú lokalizačné služby), čo je kľúčové pre praktickú časť tejto práce.
(Zdroj: `https://gs.statcounter.com/android-version-market-share`)

3.1.2 Distribúcia a ekosystém platforiem

Voľba operačného systému ovplyvňuje nielen technologický zásobník, ale aj stratégiu uvedenia na trh. Platforma Android je často preferovaná pre nasadenie minimálneho životaschopného produktu (*MVP*) kvôli flexibilnejším pravidlám v obchode Google Play v porovnaní s uzavretým ekosystémom iOS.

Hoci existujú multiplatformové riešenia, pre účely tejto práce bol zvolený natívny prístup z dôvodu maximálnej optimalizácie výkonu a prístupu k systémovým API. Porovnanie prístupov na základe analýzy [**pullur_medium_2025**] uvádza Tabuľka 5.

Tabuľka 5: Súhrnné porovnanie natívneho a multiplatformového vývoja

Metrika	**Native (Kotlin)**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Rýchlosť štartu	**Najrýchlejšia**	Stredná	Pomalšia	Rýchla
Animácie	**Plynulé**	Plynulé	Riziko lagov	–
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka

3.2 Programovací jazyk Kotlin

Kotlin je moderný, staticky typovaný jazyk, ktorý je od roku 2019 preferovanou voľbou spoločnosti Google pre vývoj aplikácií na platformu Android. Medzi jeho kľúčové prínosy pre túto prácu patria:

- **Null Safety:** Výrazne znižuje riziko pádov aplikácie (chyba *NullPointerException*) vďaka striktnému rozlišovaniu nulovateľných typov už v čase kompilácie.
- **Coroutines:** Umožňujú efektívne vykonávanie asynchrónnych úloh (napr. sťahovanie dát o podujatiach z cloudu) bez blokovania hlavného vlákna a zamŕzania používateľského rozhrania.
- **Stručnosť:** Redukcia nadbytočného kódu (*boilerplate*) umožňuje rýchlejšiu implementáciu biznis logiky odporúčacieho algoritmu.

3.3 UI Toolkit: Jetpack Compose

Na tvorbu používateľského rozhrania bol zvolený moderný deklaratívny toolkit **Jetpack Compose**. Na rozdiel od staršieho systému založeného na XML súboroch, Compose umožňuje definovať UI priamo v jazyku Kotlin.

- **Deklaratívny prístup:** Programátor opisuje výsledný stav rozhrania, pričom framework sa automaticky stará o jeho prekreslenie pri zmene dát (napr. pri aktualizácii zoznamu podujatí).
- **Komponentový dizajn:** UI je vyskladané z malých, znovupoužiteľných funkcií (*Composables*), čo uľahčuje údržbu a testovanie aplikácie.

3.4 Vývojové nástroje

Ako už sám naznačuje názov, Android Studio je oficiálne vývojové prostredie (IDE) bohaté na funkcie pre vývoj aplikácií pre Android, ktoré integruje nástroje na návrh, kódovanie, tvorbu a testovanie. Ponúka natívnu podporu pre Kotlin, Java a C++, rozsiahlu rozšíriteľnosť pluginov a bezproblémovú emuláciu naprieč typmi zariadení. Pravidelné aktualizácie zabezpečujú kompatibilitu s novými verziami systému Android, pokročilé ladenie a analýzu výkonu v jednotnom prostredí a preto je ideálnou voľbou pre tento projekt.

3.5 Technická špecifikácia a použité knižnice

3.6 Android SDK a správa verzií

Kľúčovým pilierom vývoja je **Android SDK** (*Software Development Kit*), čo je sada vývojových nástrojov poskytovaná spoločnosťou Google. SDK obsahuje knižnice, ladiace nástroje a emulátory potrebné na transformáciu zdrojového kódu do spustiteľného balíka aplikácie.

V rámci projektu je správa verzií riešená definovaním úrovní rozhraní API (*API Levels*), ktoré určujú rozsah kompatibility a dostupné systémové funkcie [**geeks_android_sdk**].

- **minSdk 26 (Android 8.0 Oreo):** Táto hodnota stanovuje najnižšiu možnú verziu systému, na ktorej je možné aplikáciu spustiť. Voľba verzie 26 bola zvolená ako optimálny kompromis medzi dostupnosťou moderných API (napr. vylepšené kanály notifikácií či správa úloh na pozadí) a širokým pokrytím trhu, ktoré aktuálne presahuje 90 % aktívnych zariadení.

- **targetSdk 35 (Android 15):** Informuje operačný systém, pre ktorú najnovšiu verziu bola aplikácia optimalizovaná a testovaná. Použitie aktuálneho SDK zabezpečuje, že aplikácia dodržiava najnovšie bezpečnostné štandardy a protokoly ochrany súkromia, ktoré systém Android vyžaduje pre moderné aplikácie v obchode Google Play.

Rozdiel medzi týmito úrovňami definuje spätnú kompatibilitu. Zatiaľ čo *minSdk* garantuje spustenie aplikácie na staršom hardvéri, *targetSdk* umožňuje systému aktivovať špecifické optimalizácie výkonu a zabezpečenia určené pre najnovšie generácie mobilných zariadení. https://www.geeksforgeeks.org/android/android-sdk-and-its-components/

3.6.1 Dizajnový systém Material Design 3

Vizuálna identita a používateľské rozhranie aplikácie sú postavené na najnovšom štandarde **Material Design 3** (známy aj ako *Material You*). Ide o adaptívny systém usmernení, komponentov a nástrojov vyvinutý spoločnosťou Google, ktorý podporuje osvedčené postupy pri navrhovaní moderných používateľských rozhraní [**material_io_m3**].

Hlavným prínosom integrácie Material Design 3 do projektu je zefektívnenie spolupráce medzi dizajnom a vývojom, čo umožňuje rýchle budovanie estetických a funkčných produktov.

- **Komponentové sady:** Material 3 poskytuje širokú škálu sád komponentov, ktoré sú špecificky navrhnuté pre rôzne prípady použitia a veľkosti obrazoviek. V projekte sú využité kľúčové prvky ako `TopAppBar` (horná lišta aplikácie), `NavigationDrawer` (bočné navigačné menu) a `Card` (karty pre prehľadné zobrazenie informácií o podujatiach).

- **Adaptivita a prístupnosť:** Systém je navrhnutý tak, aby sa automaticky prispôsoboval rôznym rozlíšeniam a orientáciám displeja, pričom kladie veľký dôraz na čitateľnosť a prístupnosť (*accessibility*) pre všetkých používateľov.

- **Integrácia s Jetpack Compose:** Využitie knižnice `androidx.compose.material3` umožňuje natívne deklaratívne definovanie štýlov, farieb a typografie priamo v kóde Kotlin, čím sa zabezpečuje vizuálna konzistencia naprieč celou aplikáciou.

3.7 Externé softvérové knižnice

Pre rozšírenie základných funkcií systému Android a zabezpečenie plynulého používateľského zážitku boli do projektu integrované špecializované knižnice. Tieto nástroje riešia kritické úlohy od navigácie v deklaratívnom prostredí až po nízkoúrovňovú komunikáciu s hardvérom kamery.

3.7.1 Navigation Compose (2.8.5)

Knižnica **Navigation Compose** poskytuje infraštruktúru pre riadenie toku obrazoviek v aplikáciách postavených na Jetpack Compose. Umožňuje definovať navigačný graf priamo v kóde Kotlin, čím nahrádza staršie riešenia založené na XML súboroch a fragmentoch.

- **NavHost:** Slúži ako vizuálny kontajner, ktorý dynamicky vymieňa jednotlivé kompozitné funkcie (*composables*) na základe aktuálnej trasy.
- **NavController:** Centrálny objekt, ktorý spravuje zásobník obrazoviek (*backstack*) a zabezpečuje správnu reakciu na systémové tlačidlo „Späť“.
- **Typová bezpečnosť:** Verzia 2.8.5 prináša natívnu podporu pre typovo bezpečnú navigáciu, čo eliminuje chyby pri prenose argumentov (napr. ID podujatia) medzi obrazovkami.

3.7.2 CameraX (1.3.4)

CameraX je moderná knižnica zameraná na prácu s fotoaparátom, ktorá odstraňuje zložitosť spojenú so správou špecifík hardvéru od rôznych výrobcov. Je postavená na koncepte prípadov použitia (*use-cases*), z ktorých boli v projekte implementované najmä:

- **Preview:** Zabezpečuje plynulý živý náhľad obrazu z kamery priamo v používateľskom rozhraní.
- **Image Analysis:** Poskytuje prístup k obrazovým dátam v reálnom čase, čo je kľúčové pre **algoritmy na skenovanie a dekódovanie QR kódov** pri vstupe na podujatia.

- **Image Capture:** Umožňuje zachytávanie fotografií (napr. pre profilové fotky alebo dokumentáciu podujatia) s automatickou optimalizáciou expozície a zaostrenia.

3.7.3 Coil (2.7.0)

Názov **Coil** predstavuje akronym pre *Coroutine Image Loader*. Ide o efektívnu knižnicu na asynchrónne načítavanie obrázkov z cloudu (Firebase Storage), ktorá je plne optimalizovaná pre prostredie Kotlin Coroutines.

- **Optimalizácia výkonu:** Knižnica automaticky vykonáva operácie ako cachovanie na disk a do pamäte RAM, zmenu mierky obrázkov (*downsampling*) a inteligentné rušenie sieťových požiadaviek pri odscrollovaní prvku z obrazovky.

- **Integrácia s Compose:** Vďaka natívnej podpore pre Jetpack Compose umožňuje zobrazenie obrázkov s minimálnym množstvom riadiaceho kódu (*boilerplate*), čím zvyšuje čitateľnosť projektu.

3.8 Architektúra projektu

V projekte využívame architektúru **MVVM** (*Model-View-ViewModel*) v kombinácii s **Repository pattern**. Tento prístup je oficiálne odporúčaný spoločnosťou Google a predstavuje súčasný štandard v Android komunite pre budovanie škálovateľných a udržiavateľných aplikácií.

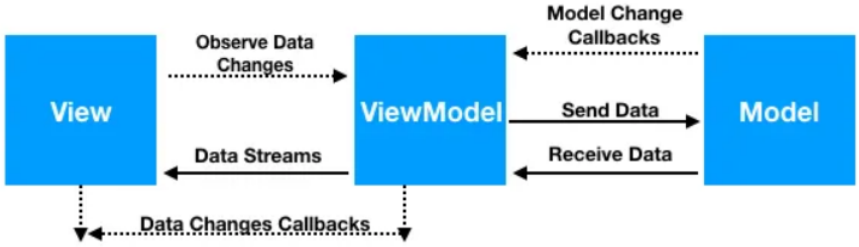
Model MVVM pozostáva z troch základných komponentov, z ktorých každý má jasne definovanú zodpovednosť:

- **Model:** Reprezentuje dátovú vrstvu a biznis logiku. V našom prípade sem patrí komunikácia s Firebase a správa lokálnych dát.

- **View (Zobrazenie):** Používateľské rozhranie, ktoré pasívne zobrazuje dáta a odosiela akcie používateľa do ViewModelu.

- **ViewModel (Model zobrazenia):** Pôsobí ako prostredník, ktorý pripravuje dáta z modelu pre zobrazenie a udržiava stav aplikácie nezávisle od životného cyklu rozhrania.

Hlavným dôvodom voľby MVVM sú jeho prínosy pre stabilitu projektu, najmä **zvýšená testovateľnosť** (možnosť testovať logiku bez UI), **jednoduchšia údržba** vďaka oddeleniu zodpovedností a **odolnosť voči zmenám konfigurácie** (napr. otočenie displeja), kedy ViewModel chráni dáta pred stratou.



Obr. 1: Diagram architektúry MVVM s Repository pattern v systéme Android

3.8.1 Moderný prístup: MVVM v kombinácii s Jetpack Compose

V súlade s princípmi čistého návrhu (*Clean Architecture*) opísanými v [**sks_medium_2024**], je v projekte implementovaný vzor MVVM s dôrazom na **jednosmerný tok dát** (*Unidirectional Data Flow - UDF*). Tradičný mechanizmus *Data Bindingu* známy zo starších verzií Androidu je tu nahradený moderným reaktívnym prístupom:

- **State Hoisting (Vynášanie stavu):** ViewModel uchováva kompletný stav obrazovky v podobe imutabilného objektu (napr. *EventsUiState*). UI komponenty sú takmer bezstavové (*stateless*), čo uľahčuje ich znovupoužiteľnosť.
- **Reaktívne pozorovanie (StateFlow):** Používateľské rozhranie je prepojené s *StateFlow* vo ViewModele. Akákoľvek zmena v dátach (napr. aktualizácia zoznamu podujatí) vyvolá okamžitú **rekompozíciu** (automatické prekreslenie) dotknutých častí rozhrania.
- **Spracovanie udalostí (Events):** Všetky akcie používateľa sú prenášané ako udalosti smerom k ViewModelu. View nikdy neupravuje dáta priamo, čo zaručuje, že *ViewModel* zostáva jediným zdrojom pravdy (*Single Source of Truth*).

3.9 Platforma Firebase

Firebase predstavuje komplexnú platformu cloudových služieb od spoločnosti Google, ktorá je navrhnutá na zrýchlenie vývoja, zvýšenie bezpečnosti a zabezpečenie škálovateľnosti moderných aplikácií. Platforma pokrýva celý životný cyklus vývoja softvéru a je využívaná svetovými lídrami ako Duolingo, NPR či Venmo.

3.9.1 Dôvody voľby a kľúčové výhody

Voľba tejto platformy pre účely mojej práce vychádza z jej schopnosti riešiť zložité vývojové výzvy pomocou integrovaných produktov:

- **Efektívna autentifikácia:** Poskytuje robustné knižnice pre správu používateľov, overovanie hesiel a integráciu poskytovateľov tretích strán (Google, GitHub atď.).
- **Škálovateľné úložisko:** Cloud Storage umožňuje efektívnu prácu s používateľským obsahom, ako sú obrázky a videá, s vysokou úrovňou zabezpečenia.
- **Real-time synchronizácia:** NoSQL databázy (Firestore, Realtime Database) umožňujú synchronizáciu dát naprieč klientmi v reálnom čase, a to aj v offline režime.
- **Bezpečnosť a správa:** Flexibilné bezpečnostné pravidlá chránia dáta pred neoprávneným prístupom priamo na úrovni cloudu.
- **Serverless logika:** Cloud Functions umožňujú rozšíriť funkcionalitu aplikácie o asynchrónne spracovanie na pozadí bez nutnosti správy vlastného servera.

3.10 Firebase Authentication

Firebase Authentication poskytuje backendové služby, ľahko použiteľné súpravy SDK a hotové knižnice používateľského rozhrania na autentifikáciu používateľov v aplikácii. Podporuje overovanie pomocou hesiel, telefónnych čísiel a populárnych poskytovateľov identít (Google, Facebook, Twitter). Služba využíva moderné priemyselné štandardy ako *OAuth 2.0* a *OpenID Connect*, pričom zabezpečuje kompletnú správu relácií a overovanie e-mailových adries.

3.10.1 Mechanizmus fungovania a správa tokenov

Proces autentifikácie vo Firebase Authentication je založený na výmene a overovaní tokenov, čo zabezpečuje bezstavovú komunikáciu medzi klientom a backendom. Celý proces prebieha v niekoľkých krokoch:

- **Prihlásenie a vytvorenie ID tokenu:** Po úspešnom overení identity (napr. zadaním hesla) Firebase servery vygenerujú *Firebase ID Token*. Ide o *JSON Web Token* (JWT), ktorý obsahuje informácie o identite používateľa (UID, e-mail, čas prihlásenia). Tento token je krátkodobý a jeho platnosť je štandardne 1 hodina.
- **Refresh Token:** Súčasne s ID tokenom klient dostane aj *Refresh Token*. Na rozdiel od ID tokenu, refresh token má dlhú platnosť. Firebase SDK ho automaticky ukladá do lokálneho úložiska prehliadača alebo zariadenia a používa ho na automatické získanie nového ID tokenu po vypršaní toho starého, čím zabezpečuje, že používateľ nemusí opakovane zadávať prihlasovacie údaje.

- **Autorizácia požiadaviek:** Pri každej požiadavke na chránené zdroje (napr. Cloud Firestore alebo vlastný backend) klientska aplikácia pripojí aktuálny ID token do hlavičky požiadavky. Prijímajúca strana následne overí integritu a platnosť tohto tokenu pomocou verejných kľúčov Google.
- **Bezpečnostné pravidlá:** Identita extrahovaná z tokenu je priamo dostupná v bezpečnostných pravidlách Firebase (*Security Rules*), čo umožňuje jednoducho definovať prístupové práva na úrovni jednotlivých používateľov (napr. `request.auth.uid == userId`).

https://firebase.google.com/docs/auth

3.10.2 Prepojenie s Cloud Firestore a dokument users/{uid}

Hoci Firebase Authentication spravuje prihlasovacie údaje, neukladá dodatočné informácie o používateľovi (napr. adresu, telefónne číslo, preferencie alebo rolu v systéme). Na tento účel sa v kombinácii s autentifikáciou využíva databáza *Cloud Firestore*.

- **Identifikátor UID:** Každý používateľ po registrácii získa unikátny identifikátor (*Unique Identifier - UID*). Tento kľúč je nemenný a slúži ako most medzi autentifikačnou službou a databázou.
- **Dokument users/{uid}:** Bežným architektonickým vzorom je vytvorenie kolekcie s názvom `users`, kde názov každého dokumentu zodpovedá konkrétnemu `uid` používateľa. Týmto spôsobom je zabezpečené, že každý prihlásený používateľ má priradený práve jeden profilový dokument.
- **Bezpečnostná integrácia:** Vďaka tomu, že sa názov dokumentu zhoduje s `uid`, je možné v *Firestore Security Rules* jednoducho definovať prístup:

```
match /users/{userId} {
  allow read, write: if request.auth != null && request.auth.uid == userId;
}
```

  Tento zápis zabezpečí, že k dokumentu v ceste `users/123` sa dostane výhradne používateľ, ktorého `uid` je `123`.

3.11 Firebase Firestore

Cloud Firestore je flexibilná, škálovateľná NoSQL databáza určená pre mobilný a webový vývoj. Udržiava dáta synchronizované medzi klientskymi aplikáciami prostredníctvom reaktívnych poslucháčov (*realtime listeners*).

Kľúčovou vlastnosťou pre túto prácu je **offline podpora**, ktorá umožňuje aplikácii plynule fungovať aj pri nestabilnom internetovom pripojení tým, že lokálne cachuje potrebné dáta a synchronizuje ich hneď po obnovení siete.

3.12 Firebase Storage

Cloud Storage pre Firebase je výkonná a nákladovo efektívna služba určená na ukladanie objektov, ako sú fotografie podujatí či profilové obrázky používateľov. Súpravy SDK zabezpečujú vysokú mieru spoľahlivosti pri nahrávaní a sťahovaní súborov bez ohľadu na kvalitu sieťového pripojenia, pričom prístup k súborom je striktne riadený integračnými pravidlami Firebase Authentication.

3.13 Technológia QR kódov

Existuje nekonenečne veľa spôsobov ako overiť vstup od podujatie, jedno z najefektívnejších je využitie QR kódov. QR kód (*Quick Response Code*) je dvojrozmerný čiarový kód, ktorý v roku 1994 vyvinula spoločnosť Denso Wave (patriaca do skupiny Toyota). V roku 2000 bol schválený ako medzinárodný štandard ISO (ISO/IEC18004). Pôvodne bol navrhnutý pre sledovanie dielov v automobilovom priemysle, no vďaka svojej efektivite sa rozšíril do všetkých oblastí digitálnej komunikácie.

3.13.1 Kľúčové vlastnosti a štruktúra

QR kód sa od klasických lineárnych čiarových kódov líši niekoľkými zásadnými charakteristikami:

- **Vysoká hustota dát:** Dokáže uchovať oveľa väčšie množstvo informácií na malej ploche (text, URL adresy, kontaktné údaje).
- **Podpora znakov:** Okrem číslic a latinky podporuje aj zložitejšie znakové sady (napr. binárne dáta alebo Kanji).
- **Odolnosť voči poškodeniu:** Vďaka vstavanej korekcii chýb (Reed-Solomonov kód) je možné QR kód správne prečítať, aj keď je časť jeho plochy znečistená alebo poškodená.

- **Otvorený štandard:** Patent na túto technológiu bol uvoľnený pre verejnosť, čo znamená, že QR kódy môže generovať a používať ktokoľvek bezplatne.

3.13.2 Využitie v mobilných zariadeniach

V súčasnosti je väčšina moderných smartfónov vybavená kamerami s natívnou podporou čítania QR kódov. Tento mechanizmus sa najčastejšie využíva na:

- **Rýchly prístup k informáciám:** Automatické presmerovanie prehliadača na zakódovanú URL adresu.
- **Bezkontaktné interakcie:** Elektronické vstupenky, platby (napr. skenovanie faktúr) alebo rýchle pripájanie k Wi-Fi sieťam.
- **Autentifikáciu:** Prepojenie mobilnej aplikácie s webovým rozhraním (napr. WhatsApp Web).

(Zdroj: `https://www.iso.org/obp/ui/en/#iso:std:iso-iec:18004:ed-4:v1:en`)

ZXing Core (3.5.3) *ZXing* (skratka pre „Zebra Crossing“) je široko používaná open-source knižnica na prácu s čiarovými kódmi [**zxing_repo**]. V rámci aplikácie sa využíva jej jadro (*Core*) na **generovanie QR kódov**.

- **Proces:** Knižnica transformuje unikátny identifikačný reťazec (token) z databázy do matice bitov (*BitMatrix*), ktorá je následne vykreslená ako bitmapový obrázok na displeji zariadenia.
- **Konfigurácia:** Umožňuje presné nastavenie kódovania (UTF-8) a úrovne korekcie chýb, čo zabezpečuje čitateľnosť kódu aj pri nižšom jase displeja.

https://github.com/zxing/zxing

ML Kit Barcode Scanning (17.3.0) Pre reverzný proces – dekódovanie QR kódu – bola zvolená moderná knižnica od spoločnosti Google v rámci sady *ML Kit* [**mlkit_barcode**]. Na rozdiel od starších knižníc využíva modely **strojového učenia** na detekciu kódov v reálnom čase.

- **Výkon:** Algoritmus dokáže identifikovať a dekódovať QR kód takmer okamžite, a to aj v prípade, že je kód naklonený, rozmazaný alebo sa nachádza v pohybe.
- **Integrácia:** Knižnica úzko spolupracuje s rozhraním *CameraX*, kde spracováva prúd obrazových dát (*Image Analysis*) a v prípade nálezu kódu vráti jeho textovú hodnotu pre ďalšiu validáciu voči systému Firebase.

https://developers.google.com/ml-kit/vision/barcode-scanning

3.14 Cloudová infraštruktúra a Google Maps Platform

Aplikácia využíva služby **Google Cloud Platform (GCP)**, čo je komplexný súbor cloudových služieb poskytovaných spoločnosťou Google. GCP umožňuje vývojárom a podnikom prevádzkovať aplikácie na bezpečnej, vysoko výkonnej a škálovateľnej infraštruktúre, ktorá je identická s tou, ktorú Google používa pre svoje vlastné produkty. https://www.geeksforgeeks.org/devops/google-cloud-platform-gcp/

3.14.1 Globálna infraštruktúra: Regióny a Zóny

Pre zabezpečenie nízkej latencie a vysokej dostupnosti geolokačných dát (máp a miest) využíva GCP hierarchickú štruktúru dátových centier:

- **Zóny (Zones):** Predstavujú základnú úroveň, kde sú nasadené výpočtové prostriedky a úložiská. Hoci sa zóna často vizualizuje ako dátové centrum, v skutočnosti môže zahŕňať viacero fyzických budov so spoločným sieťovým prepojením.
- **Regióny (Regions):** Sú nezávislé geografické oblasti (napr. *europe-west2*), ktoré zoskupujú viacero zón. Latencia medzi zónami v rámci jedného regiónu je spravidla pod 5 milisekúnd.
- **Multi-Region:** Vybrané služby (napr. Google Cloud Storage) podporujú ukladanie dát redundantne v minimálne dvoch geografických lokalitách vzdialených aspoň 160 km, čo zabezpečuje odolnosť voči rozsiahlym prírodným katastrofám.

Využitie tejto infraštruktúry zaručuje, že mapové podklady v aplikácii sú načítavané z najbližšieho možného uzla, čo minimalizuje odozvu pri interakcii používateľa s mapou.

https://www.geeksforgeeks.org/devops/google-cloud-platform-gcp/

Maps SDK for Android *Maps SDK for Android* umožňuje pridávať mapové podklady do aplikácie (vrátane Wear OS) s využitím údajov Google Maps, zobrazenia máp a reakcií na gestá. SDK poskytuje nástroje na vkladanie doplňujúcich informácií o polohách a podporuje interakciu používateľa prostredníctvom značiek (*Markers*), polygónov a iných prekrytí (*Overlays*).

Platforma plne podporuje programovacie jazyky Kotlin a Java, pričom poskytuje dodatočné knižnice a rozšírenia pre pokročilé funkcie a moderné programovacie techniky.

Places SDK for Android *Places SDK for Android* umožňuje vytvárať aplikácie, ktoré kontextuálne reagujú na okolité podniky a iné miesta v blízkosti zariadenia. Tento nástroj dopĺňa základné geografické služby systému Android o bohatú databázu miest, ktoré majú pre používateľa konkrétny význam.

Kľúčové funkcie a rozhrania zahŕňajú:

- **Programový prístup k databáze:** SDK poskytuje prístup k informáciám o miestnych podnikoch, bodoch záujmu a aktuálnej polohe zariadenia.
- **Autocomplete:** Ponúka predpripravené komponenty (*Widgets*) na predpovedanie názvov miest v reakcii na vyhľadávacie dopyty používateľa.
- **Definícia miesta (*Place*):** Miesto je v rámci API reprezentované rozhraním *Place*. Definujeme ho ako fyzický priestor s názvom (napr. firmy, pamiatky, geografické polohy). Objekt *Place* obsahuje údaje ako názov, adresa, ID miesta, geografická poloha, telefónne číslo, typ miesta či webová adresa.

Podobne ako Maps SDK, aj tento nástroj plne podporuje jazyky Kotlin a Java vrátane rozšírení pre pokročilý vývoj.

Google Sign-In Tento komponent spravuje autentifikačný tok *OAuth 2.0* a životný cyklus prístupových tokenov. Integruje prihlasovanie pomocou Google účtu, čím zjednodušuje overovanie identity používateľa a umožňuje bezpečnú komunikáciu s ostatnými Google rozhraniami (API) bez nutnosti manuálnej správy prihlasovacích údajov.

Google Location Services Knižnica `play-services-location` je súčasťou služieb Google Play a poskytuje vysoko presné a energeticky efektívne sledovanie polohy zariadenia. V aplikácii sa využíva na získanie aktuálnych súradníc používateľa, čo je kľúčové pre funkcie ako zobrazovanie najbližších podujatí alebo navigáciu k miestu konania.

Google Location Services Knižnica `play-services-location` je kľúčovou súčasťou služieb Google Play, ktorá poskytuje rozhrania pre prácu s polohou zariadenia. V projekte sa využíva najmä *Fused Location Provider API*, ktoré inteligentne kombinuje dáta z GPS, Wi-Fi sietí a mobilných vysielačov.

Konfigurácia a bezpečnosť rozhraní Prepojenie aplikácie s cloudovou infraštruktúrou je realizované prostredníctvom konfiguračného súboru `google-services.json` a unikátnych API kľúčov. Bezpečnosť týchto rozhraní je zabezpečená na dvoch úrovniach:

- **Technické obmedzenia:** Prístup k API je v *Google Cloud Console* striktne obmedzený na balík aplikácie (*Package Name*) a unikátny **SHA-1 odtlačok** podpisového certifikátu vývojára.
- **Separácia citlivých údajov:** API kľúče sú uložené mimo zdrojového kódu v súbore `local.properties`, ktorý je ignorovaný systémom správy verzií, a do projektu sú načítavané pomocou nástroja *Secrets Gradle Plugin*.

3.15 Cloudinary – správa digitálnych aktív

Pre efektívnu správu, optimalizáciu a doručovanie vizuálneho obsahu (obrázky podujatí, logá, profilové fotografie) bola do projektu integrovaná platforma **Cloudinary**. Ide o moderný systém správy digitálnych aktív (DAM – *Digital Asset Management*), ktorý využíva umelú inteligenciu a pokročilé algoritmy na spracovanie obrazu [**cloudinary_blog**].

3.15.1 Kľúčové výhody a funkcie

Cloudinary poskytuje nástroje, ktoré presahujú možnosti bežného cloudového úložiska:

- **Automatická optimalizácia:** Systém pomocou AI automaticky upravuje formát, kvalitu a rozlíšenie obrázkov v reálnom čase podľa zariadenia používateľa, čo výrazne znižuje spotrebu dát a zrýchľuje načítavanie aplikácie.
- **Dynamické transformácie:** Umožňuje vykonávať úpravy (orezanie na štvorec, zmena veľkosti, pridanie vodoznaku) priamo prostredníctvom úpravy URL adresy obrázka, bez nutnosti manuálneho generovania viacerých verzií súborov.
- **Škálovateľnosť:** Infraštruktúra je navrhnutá tak, aby bezchybne doručovala vizuálny obsah vo veľkom meradle (*at scale*) pri zachovaní vysokej rýchlosti odozvy.

3.15.2 Porovnanie s Firebase Storage a ekonomické hľadisko

Hoci ekosystém Firebase, využívaný v tejto práci, ponúka vlastnú službu *Firebase Storage*, pre účely správy obrázkov bola zvolená platforma Cloudinary z nasledujúcich dôvodov:

1. **Nákladová efektivita:** Firebase Storage uplatňuje striktné limity na prenos dát a ukladanie v bezplatnom balíku (*Spark Plan*), pričom pokročilé funkcie spracovania obrazu sú často spojené s dodatočnými nákladmi v rámci plateného programu *Blaze*. Cloudinary ponúka robustný bezplatný program, ktorý zahŕňa komplexné AI transformácie bez nutnosti okamžitých investícií.
2. **Absencia natívnej manipulácie:** Firebase Storage slúži primárne ako „pasívne“ úložisko súborov. Na dosiahnutie funkcií, ktoré má Cloudinary natívne (napr. zmena veľkosti na strane servera), by bolo vo Firebase potrebné implementovať a platiť za *Cloud Functions*, čo by zvýšilo komplexnosť aj cenu riešenia.

3.16 Návrh a evolúcia algoritmu odporúčania

Pôvodný návrh systému sa spoliehal výhradne na metódu filtrovania založeného na obsahu (*Content-based filtering*). Táto metóda predpokladala, že budúci záujem používateľa je priamo úmerný jeho historickej interakcii s konkrétnymi kategóriami podujatí.

Prechod na hybridný model Počas vývoja a testovania sa ukázalo, že čisto historické dáta nemusia reflektovať aktuálny zámer používateľa (napríklad ak používateľ v minulosti navštevoval športové podujatia, ale momentálne hľadá kultúrny program). Z tohto dôvodu sme prešli na **váhový hybridný systém (Weighted Hybrid System)**, ktorý kombinuje:

- **Implicitné dáta:** História navštívených podujatí a automaticky generovaný profil preferencií.
- **Explicitné dáta:** Priame nastavenie preferencií používateľom v reálnom čase pomocou váhových koeficientov (slidery).

Zdôvodnenie voľby Hlavným argumentom pre túto zmenu bola skutočnosť, že podujatia (*eventy*) obsahujú fixné atribúty (kategória, podkategória, poloha), ktoré existujú nezávisle od histórie používateľa. Hybridný systém umožňuje tieto fixné atribúty dynamicky prepojiť s aktuálnou potrebou používateľa, čím sa eliminuje problém „informačnej bubliny“, do ktorej by čistý *Content-based* systém používateľa uzatvoril.

4 Návrh riešenia

V tejto sekcii sa budeme venovať samotnému návrhu riešenia našej aplikácie začínajúc od funkčných a nefunkčných požiadaviek, cez grafické návrhy a diagrami až po samotnú implemntáciu a testovanie

4.0.1 Funkcionálne a Nefunkcionálne požiadavky

Definivanie funkcionálnych a nefunkcionálnych požiadaviek je podstatným krokom návrhu riešenia, pretože nám pomáha jasne stanoviť správanie aplikácie ktoré by splnili jej cieľ

4.0.2 Funkcionálne požiadavky

Funkcionálne požiadavky definujú správanie systému a služby, ktoré používateľ od systému očakáva. Ich hlavným cieľom je:

1. Definovať funkcie, ktoré sa od systému očakávajú.
2. Napomôcť k lepšiemu pochopeniu funkcionality systému.
3. Identifikovať, čo systém musí alebo nesmie vykonávať.
4. Umožniť fungovanie systému (funkčnosť nie je priamo závislá od nefunkcionálnych požiadaviek).
5. Zabezpečiť, že systém bude spĺňať reálne potreby používateľov.
6. Definovať prvky nevyhnutné pre samotný chod systému.
7. Stanoviť povinné body, ktoré musia byť v systéme implementované.
8. Zdokumentovať požiadavky, ktoré sú zvyčajne definované priamo používateľom.
9. Umožniť vizualizáciu a pochopenie procesov pomocou prípadov použitia (*Use Case*).

Medzi konkrétne funkcionálne požiadavky navrhovanej aplikácie patrí:

- registrácia a prihlásenie (login) používateľov,
- verifikácia používateľa pri prihlasovaní,
- možnosť vytvorenia novej udalosti,

- pridávanie podrobností k udalosti (fotografie, popis, miesto konania, čas, typ udalosti, maximálny počet účastníkov a pod.),
- prehliadanie dostupných udalostí na interaktívnej mape,
- nastavenie polomeru (kruhu) pre zobrazovanie udalostí v okolí používateľa,
- filtrovanie udalostí podľa preferencií,
- správa účastníkov (možnosť odstrániť účastníka z vlastnej udalosti),
- ukladanie vybraných udalostí do zoznamu obľúbených,
- zobrazenie histórie (navštívené, odporúčané a nastávajúce udalosti),
- možnosť potvrdenia účasti na udalosti,
- komunikácia s ostatnými účastníkmi podujatia,
- hodnotenie udalostí,
- zdieľanie (*sharing*) podujatí,
- prehliadanie profilov ostatných používateľov,
- pridávanie priateľov,
- správa a nastavenie notifikácií,
- systém nahlasovania nevhodných používateľov alebo udalostí,
- možnosť zmeny jazyka v prostredí aplikácie,
- automatizovaný poradovník náhradníkov (*Waitlist*),
- systém penalizácie alebo zablokovania používateľa, ktorý potvrdil účasť, nedostavil sa a včas sa neodhlásil,
- odporúčací systém,
- možnosť hodnotenia udalostí,
- možnosť odísť z udalosti.

4.0.3 Nefunkcionálne požiadavky

Nefunkcionálne požiadavky definujú kvalitatívne vlastnosti systému a obmedzenia, za ktorých musí systém pracovať. Ich charakteristika je nasledovná:

1. Vysvetľujú vlastnosti produktu a jeho atribúty kvality.
2. Napomáhajú k pochopeniu výkonu a správania systému.
3. Identifikujú spôsob, akým má systém stanovené úlohy vykonávať.
4. Samotné nefunkcionálne požiadavky bez funkčných nepostačujú na chod systému.
5. Zabezpečujú, že systém bude spĺňať špecifické očakávania používateľov na kvalitu.
6. Sú žiaduce pre celkový zážitok, hoci v niektorých prípadoch nie sú kritické pre základnú funkcionalitu.
7. Ich splnenie zvyšuje konkurencieschopnosť a stabilitu softvéru.
8. Zvyčajne ich definujú softvéroví inžinieri, vývojári alebo architekti na základe technických potrieb.
9. Možno ich dokumentovať a chápať ako atribúty kvality (napr. bezpečnosť, rýchlosť).

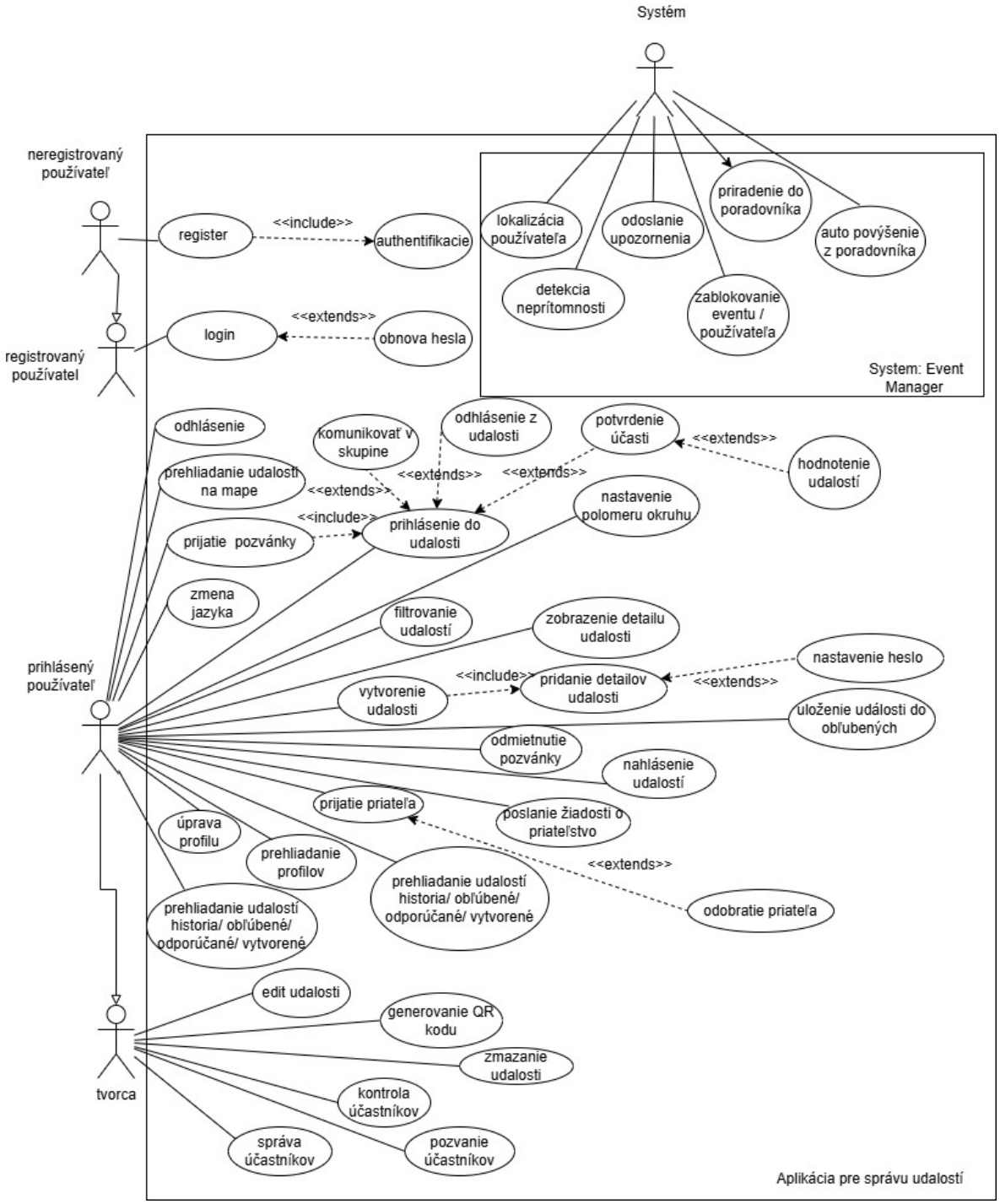
Pre navrhovanú aplikáciu boli definované tieto konkrétne nefunkcionálne požiadavky:

- **Intuitívnosť:** Používateľské rozhranie musí byť navrhnuté tak, aby bola navigácia v aplikácii jasná aj bez dodatočnej dokumentácie.
- **Responzivita:** Rozhranie aplikácie sa musí korektne prispôsobovať rôznym rozlíšeniam a pomerom strán Android zariadení.
- **Rýchlosť odozvy:** Systém musí zabezpečiť rýchlu odozvu pri načítavaní udalostí z mapy a pri spracovaní dát v odporúčacom algoritme (do 2 sekúnd).
- **Stabilita a funkčnosť:** Systém musí garantovať spoľahlivé prihlásenie používateľa a stabilitu procesov na pozadí.
- **Spracovanie a zaznamenávanie dát:** Efektívne ukladanie dát do databázy Firebase Firestore so zreteľom na minimalizáciu dátových prenosov.
- **Zaznamenávanie polohy:** Presné a šetrné spracovanie GPS súradníc používateľa pre potreby mapy a *Check-inu*.

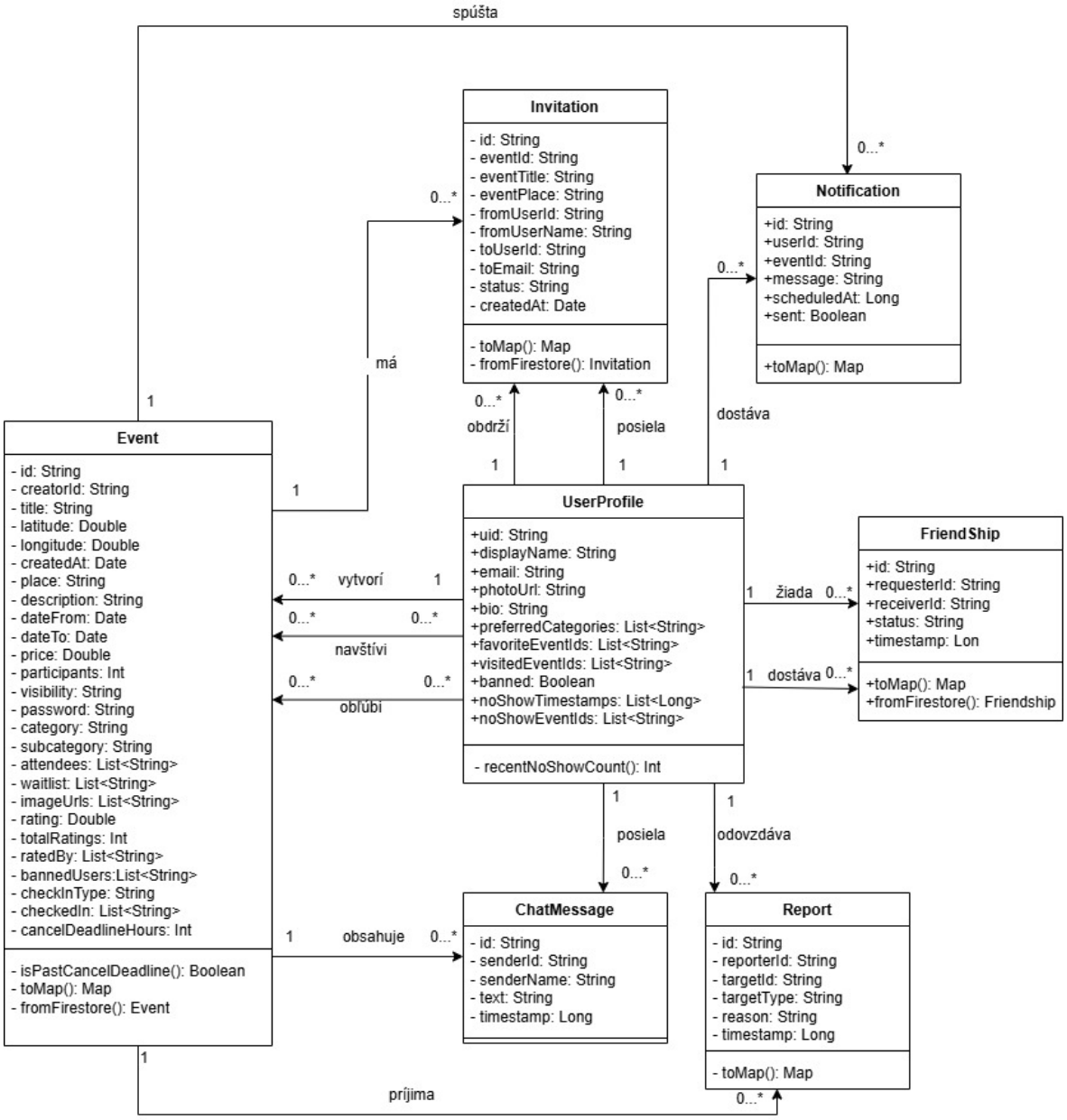
- **Správa povolení:** Systém musí korektne vyžadovať a spracovávať povolenia od používateľa (prístup k polohe, galérii a kalendáru).
- **Ochrana údajov:** Zabezpečenie osobných údajov používateľov a šifrovaná komunikácia medzi klientom a serverom.
- **Kompatibilita:** Možnosť inštalácie a synchrónneho používania na viacerých Android zariadeniach pod jedným účtom.
- **Rozšíriteľnosť:** Architektúra kódu musí umožňovať jednoduché pridávanie nových funkcií v budúcnosti bez nutnosti prepisovania jadra aplikácie.

4.0.4 UML diagramy

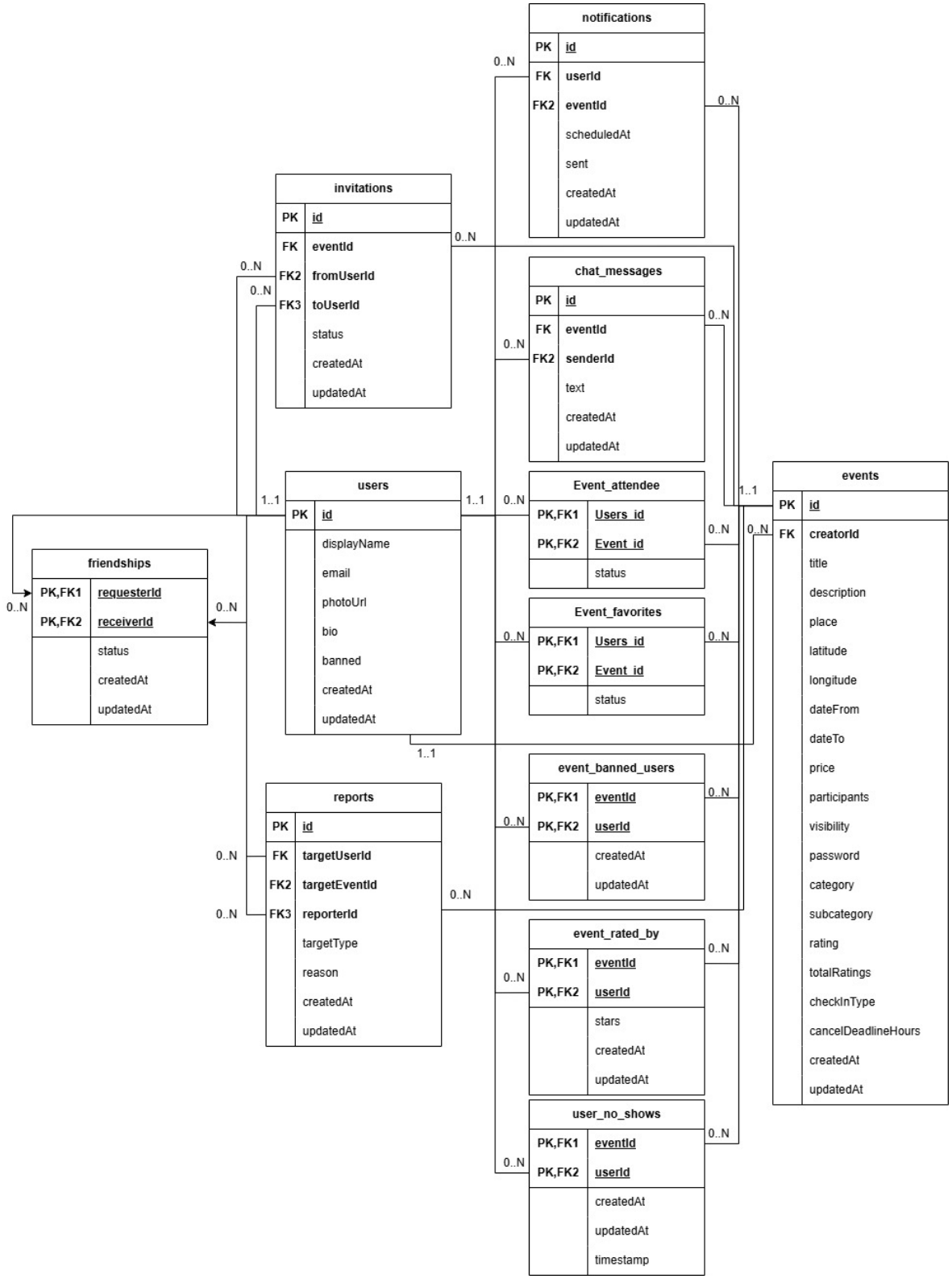
V tejto časti sú znázornené UML diagramy navrhovanej aplikácie, ktoré vizualizujú jej štruktúru a správanie.



Obr. 2: Diagram prípadov použitia (Use Case diagram)



Obr. 3: Diagram tried (Class diagram)



Obr. 4: Relačná databáza

4.1 Implementacia

V tejto časti sa zameriame na samotnú implementáciu navrhovanej aplikácie, ktorá zahŕňa výber technológií, architektúru systému a detailný popis jednotlivých komponentov.

4.1.1 Autentifikácia a správa používateľov

Autentifikačný proces v aplikácii je postavený na službe *Firebase Authentication*. Systém podporuje prihlásenie prostredníctvom e-mailu a hesla, ako aj integráciu s účtom Google. Implementácia striktne dodržiava architektúru **MVVM** (Model-View-ViewModel), čím oddeľuje logiku volania API od používateľského rozhrania.

Architektonické rozdelenie:

- **AuthService:** Dátová vrstva, ktorá priamo komunikuje s Firebase SDK (metódy ako `signInWithEmailAndPassword` alebo `createUserWithEmailAndPassword`).
- **UseCases** (napr. `LoginUserUseCase`): Doménová vrstva, ktorá zapuzdruje biznis logiku a pripravuje dáta pre ViewModel.
- **ViewModels** (`LoginViewModel`, `RegisterViewModel`): Prezentačná vrstva, ktorá spravuje stav používateľského rozhrania (indikátor načítavania `isLoading`, chybové hlásenia).
- **LoginScreen:** UI vrstva vytvorená pomocou Jetpack Compose.

Implementácia kľúčových funkcií: Služba `AuthService` využíva asynchrónne volania pomocou Kotlin korutín. Registrácia nového používateľa zahŕňa nielen vytvorenie účtu, ale aj okamžitú aktualizáciu profilu o zobrazované meno (*displayName*):

```kotlin
suspend fun registerWithEmail(
    email: String,
    password: String,
    displayName: String
): FirebaseUser? {
    val credential = auth.createUserWithEmailAndPassword(email, password).await()
    credential.user?.updateProfile(
        userProfileChangeRequest { this.displayName = displayName }
    )?.await()
    return credential.user
}
```

Výpis kódu 1: Registrácia používateľa v AuthService

Prihlásenie cez Google účet prebieha prostredníctvom `GoogleSignInClient`, pričom po získaní `idToken` sa volá `firebaseAuthWithGoogle`:

```
suspend fun firebaseAuthWithGoogle(idToken: String): FirebaseUser? {
    val credential = GoogleAuthProvider.getCredential(idToken, null)
    val result = auth.signInWithCredential(credential).await()
    return result.user
}
```

Výpis kódu 2: Prihlásenie cez Google v AuthService

alebo normálne prihlásenie pomocou e-mailu a hesla:

```
suspend fun signInWithEmail(email: String, password: String) {
    auth.signInWithEmailAndPassword(email, password).await()
}
```

Výpis kódu 3: Prihlásenie pomocou e-mailu a hesla

Validácia vstupných údajov: Pred odoslaním požiadavky na server aplikácia vykonáva lokálnu validáciu na strane klienta, čím šetrí sieťovú prevádzku a zlepšuje odozvu rozhrania. Kontroluje sa:

- Formát e-mailovej adresy (prítomnosť znaku `@`).
- Komplexnosť hesla — minimálna dĺžka 6 znakov, aspoň jedno veľké písmeno a aspoň jeden špeciálny znak.
- Zhoda hesla s jeho potvrdením pri registrácii.

```
passwordError = when {
    password.length < 6              -> "Heslo musi mat aspon 6 znakov."
    !password.any { it.isUpperCase() } -> "Heslo musi obsahovat aspon jedno velke
    pismeno."
    !password.any { !it.isLetterOrDigit() } -> "Heslo musi obsahovat aspon jeden
    specialny znak."
    else -> null
}
```

Výpis kódu 4: Validácia hesla v RegisterViewModel

Bezpečnostné obmedzenia a banovanie: Nad rámec štandardnej autentifikácie aplikácia implementuje kontrolu stavu používateľa. Pri každom prihlásení sa overuje atribút `banned` v dokumente používateľa v databáze Firestore. V prípade, že je používateľ zablokovaný, aplikácia ho okamžite odhlási a zamedzí prístup k chránenému obsahu.

4.2 Implementácia mapového modulu a geolokalizácie

Ako základ pre mapu je použitá knižnica *Google Maps SDK*, ktorá poskytuje všetky potrebné funkcionality pre prácu s geografickými dátami. Používateľ na hlavnej obrazovke vidí interaktívnu mapu, na ktorej sú jednotlivé udalosti zobrazené vo forme markerov.

Zobrazenie mapy a integrácia s Compose Keďže `MapView` nie je natívny komponent prostredia *Jetpack Compose*, v aplikácii je využitý `AndroidView`, ktorý slúži ako most (*bridge*) medzi klasickým *Android View* systémom a *Compose UI*. Mapa sa pri prvotnom načítaní predvolene centruje na súradnice Bratislavy.

```
val mapView = remember { MapView(context).also { it.onCreate(null) } }

AndroidView(factory = { mapView }, modifier = Modifier.fillMaxSize()) { mv ->
    mv.getMapAsync { map ->
        map.moveCamera(CameraUpdateFactory.newLatLngZoom(BRATISLAVA, 13f))
    }
}
```

Výpis kódu 5: Inicializácia MapView v prostredí Compose

`MapView` disponuje vlastným životným cyklom, ktorý je potrebné manuálne prepojiť s životným cyklom obrazovky pomocou `DisposableEffect`. Tým sa zabezpečí, že mapa správne reaguje na udalosti ako *onPause*, *onResume* a *onDestroy*.

GPS poloha a správa oprávnení Aplikácia vyžaduje oprávnenie `ACCESS_FINE_LOCATION`, po ktorého udelení získa poslednú známu polohu zariadenia pomocou `FusedLocationProviderClient`. Kamera mapy sa presunie na polohu používateľa iba raz pri inicializácii. Tento stav je riadený príznakom `mapMovedToUser` v `AppViewModel`, čím sa zabraňuje nežiaducemu a opakovanému presunu kamery pri každej rekomponzícii rozhrania.

```
LocationServices.getFusedLocationProviderClient(context)
    .lastLocation
    .addOnSuccessListener { loc ->
        userLocation = LatLng(loc.latitude, loc.longitude)
        appViewModel.updateUserLocation(loc.latitude, loc.longitude)
    }
```

Výpis kódu 6: Získanie aktuálnej polohy používateľa

Práca s markermi a interakcia Všetky udalosti sú konvertované na objekty `MarkerOptions` pomocou metódy `toMarkerOptions()`, ktorá je definovaná priamo v dátovom modeli `Event`.

```
fun toMarkerOptions(icon: BitmapDescriptor? = null): MarkerOptions =
    MarkerOptions()
```

```
        .position(LatLng(latitude, longitude))
        .title(title)
        .apply { icon?.let { icon(it) } }
```

Výpis kódu 7: Konverzia modelu Event na MarkerOptions

Zobrazované markery sú synchronizované so stavom `appViewModel.events` prostredníctvom `snapshotFlow`. Pri každej zmene zoznamu udalostí sa mapa reaktívne aktualizuje a pridá nové body s vlastným grafickým vizuálom (čierny pin). Do vlastnosti `tag` každého markera sa ukladá unikátne `event.id`, čo umožňuje presnú identifikáciu udalosti pri interakcii. Po kliknutí na marker aplikácia identifikuje udalosť a naviguje používateľa na obrazovku s detailom.

Okruh pôsobnosti a filtrovanie Pre lepšiu prehľadnosť aplikácia filtruje udalosti na základe ich vzdialenosti od používateľa. Markery nachádzajúce sa mimo nastaveného okruhu sú automaticky skryté (`marker.isVisible = false`). Tento okruh je na mape vizualizovaný ako žltý polopriesvitný kruh (*Circle*), ktorého polomer je prepojený s nastaveniami v `AppViewModel`.

Personalizácia zobrazenia Používateľ má možnosť prepínať medzi piatimi štýlmi mapového podkladu: *NORMAL, SATELLITE, HYBRID, TERRAIN* a špeciálnym *DARK* režimom, ktorý využíva vlastnú definíciu štýlu vo formáte JSON.

4.3 Eventy — implementácia

4.3.1 Tvorba eventu

Obrazovka tvorby eventu (`EventCreationScreen.kt`) sa otvorí po potvrdení polohy na mape. Formulár obsahuje nasledujúce polia:

- **Názov a popis** — textové polia, obe sú povinné.
- **Miesto** — predvyplnené adresou získanou z Geocoderu / Places API.
- **Cena a počet účastníkov** — hodnota 0 znamená zadarmo, resp. neobmedzený počet.
- **Dátum a čas** — výber cez `DatePickerDialog` a `TimePickerDialog`.
- **Trvanie** — zadávané v hodinách a minútach; hodnota `dateTo` sa vypočíta automaticky:

```
val computedDateTo = dateFrom?.let {
    Date(it.time + totalMinutes * 60 * 1000L)
}
```

- **Kategória a podkategória** — výber z rozbaľovacieho zoznamu definovaného v `CATEGORY_MAP`.
- **Viditeľnosť** — verejné alebo súkromné; súkromný event vyžaduje heslo.
- **Typ check-in** — jedna zo štyroch možností: `none`, `qr`, `manual`, `qr_and_manual`.
- **Deadline odhlásenia** — počet hodín pred eventom, po uplynutí ktorých sa odhlásenie penalizuje.

Upload obrázkov cez Cloudinary: Obrázky sa vyberajú z galérie zariadenia pomocou `ActivityResultContracts.GetMultipleContents()`. Každý obrázok sa uploaduje ako `multipart/form-data` priamo na Cloudinary API bez potreby vlastného backendu:

```
private suspend fun uploadToCloudinary(
    context: Context,
    uri: Uri,
    eventId: String
): String {
    val url = URL("https://api.cloudinary.com/v1_1/$cloudName/image/upload")
    val conn = (url.openConnection() as HttpsURLConnection).apply {
        requestMethod = "POST"
        setRequestProperty(
            "Content-Type",
            "multipart/form-data; boundary=$boundary"
        )
    }
    // zapíše upload_preset + bytes obrázka
    // vráti secure_url z JSON odpovede
}
```

Výpis kódu 8: Upload obrázka na Cloudinary

Prístupové kľúče (`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_UPLOAD_PRESET`) sú uložené v súbore `local.properties` a sprístupnené cez `BuildConfig`.

Uloženie eventu: Po vyplnení formulára sa event uloží volaním `onSave()`:

```
onSave(event.copy(
    title               = title.trim(),
    dateFrom            = dateFrom,
    dateTo              = computedDateTo,
    visibility          = if (isPrivate) "private" else "public",
    imageUrls           = existingUrls + uploadedUrls,
    checkInType         = checkInType,
```

```
    cancelDeadlineHours = cancelDeadlineInput.toIntOrNull() ?: 0
))
```

Výpis kódu 9: Uloženie eventu

Metóda `AppViewModel.saveEventToMap()` uloží event súčasne do in-memory mapy `_events` aj do databázy Firestore:

```
fun saveEventToMap(event: Event) {
    _events[event.id] = event
    viewModelScope.launch {
        db.collection("events")
          .document(event.id)
          .set(event.toMap())
          .await()
    }
}
```

Výpis kódu 10: Uloženie eventu do Firestore

4.3.2 Detail eventu

Obrazovka detailu eventu (`EventDetailScreen.kt`) je rozdelená do troch tabov.

Tab 0 — Detail: Zobrazuje všetky informácie o evente. Kľúčové stavy používateľa sú určené nasledovne:

```
val isCreator    = event.creatorId == currentUserId
val isAttending  = currentUserId in event.attendees
val isOnWaitlist = currentUserId in event.waitlist
val isFull       = event.participants > 0 &&
                   event.attendees.size >= event.participants
val isBanned     = currentUserId in event.bannedUsers
```

Výpis kódu 11: Stavové premenné v detaile eventu

Tlačidlá akcie sa dynamicky menia podľa stavu — tvorca eventu vidí možnosti úpravy a mazania, prihlásený účastník vidí odhlásenie, ostatní používatelia vidia možnosť prihlásenia alebo zaradenia do poradovníka.

Tab 1 — Účastníci: Zobrazuje zoznam účastníkov s ich avatarmi. Tvorca eventu má navyše možnosť:

- manuálne označiť príchod účastníka (ak `checkInType == "manual"`),
- odstrániť účastníka z eventu — odstránený účastník je zároveň zaradený do zoznamu `bannedUsers` a prvý používateľ z poradovníka je automaticky povýšený.

Tab 2 — Chat: Real-time chat implementovaný pomocou Firestore `snapshotListener`. Prístup do chatu majú výlučne prihlásení účastníci a tvorca eventu.

4.3.3 Správa účasti

Správa účasti je implementovaná v `AppViewModel` prostredníctvom troch metód.

joinEvent — prihlásenie na event: Ak je event plný, používateľ je zaradený do poradovníka; v opačnom prípade priamo medzi účastníkov:

```
fun joinEvent(eventId: String, userId: String) {
    val isFull = event.participants > 0 &&
                 event.attendees.size >= event.participants
    val updated = if (isFull)
        event.copy(waitlist  = event.waitlist  + userId)
    else
        event.copy(attendees = event.attendees + userId)
    // uloží do Firestore
}
```

Výpis kódu 12: Prihlásenie na event

leaveEvent — odhlásenie z eventu: Pri odhlásení sa automaticky povýši prvý používateľ z poradovníka. Ak sa používateľ odhlási po uplynutí deadlinu, je mu zaznamenaný no-show:

```
fun leaveEvent(eventId: String, userId: String) {
    val applyPenalty = event.isPastCancelDeadline()
    var updated = event.copy(attendees = event.attendees - userId)
    if (updated.waitlist.isNotEmpty()) {
        val next = updated.waitlist.first()
        updated = updated.copy(
            attendees = updated.attendees + next,
            waitlist  = updated.waitlist.drop(1)
        )
    }
    if (applyPenalty) recordNoShow(userId, eventId)
}
```

Výpis kódu 13: Odhlásenie z eventu

removeAttendee — odstránenie účastníka tvorcom: Odstránený účastník je zároveň zablokovaný pre daný event a prvý z poradovníka je automaticky povýšený:

```
fun removeAttendee(eventId: String, attendeeId: String) {
    val updated = event.copy(
        attendees   = event.attendees   - attendeeId,
```

```
        bannedUsers = event.bannedUsers + attendeeId
    )
    // + automatické povýšenie z poradovníka
}
```

Výpis kódu 14: Odstránenie účastníka tvorcom eventu

4.3.4 Filtrovanie udalostí

Filtrovanie je implementované ako samostatná servisná trieda `EventFilterService`, ktorá prijíma kritériá a aplikuje ich na zoznam eventov.

FilterCriteria: Dátová trieda zapuzdruje všetky dostupné filtre:

```
data class FilterCriteria(
    val name           : String? = null,
    val category       : String? = null,
    val subcategory    : String? = null,
    val participants   : Int?    = null,
    val maxPrice       : Double? = null,
    val dateFrom       : Date?   = null,
    val dateTo         : Date?   = null,
    val maxDistanceKm  : Double? = null,
    val refLat         : Double  = 0.0,
    val refLng         : Double  = 0.0
)
```

Výpis kódu 15: Dátová trieda FilterCriteria

EventFilterService: Trieda aplikuje kritériá postupne — ak event nevyhovuje čo i len jednému z nich, je zo zoznamu vyradený:

```
class EventFilterService(private val criteria: FilterCriteria) {

    fun filter(allEvents: List<Event>): List<Event> =
        allEvents.filter { matches(it) }

    private fun matches(event: Event): Boolean {
        if (!criteria.name.isNullOrEmpty() &&
            !event.title.lowercase().contains(criteria.name.lowercase()))
            return false

        if (!criteria.category.isNullOrEmpty() &&
            event.category != criteria.category)
            return false

```

```
15         if (!criteria.subcategory.isNullOrEmpty() &&
16             event.subcategory != criteria.subcategory)
17             return false
18
19         if (criteria.participants != null && criteria.participants > 0 &&
20             event.participants < criteria.participants)
21             return false
22
23         if (criteria.maxPrice != null && criteria.maxPrice < 500.0 &&
24             event.price > criteria.maxPrice)
25             return false
26
27         if (criteria.dateFrom != null && event.dateFrom != null &&
28             event.dateFrom.before(criteria.dateFrom))
29             return false
30
31         if (criteria.maxDistanceKm != null) {
32             val dist = haversineKm(
33                 criteria.refLat, criteria.refLng,
34                 event.latitude,  event.longitude
35             )
36             if (dist > criteria.maxDistanceKm) return false
37         }
38         return true
39     }
40 }
```

Výpis kódu 16: Implementácia EventFilterService

Výpočet vzdialenosti — Haversine: Na výpočet vzdialenosti medzi dvoma GPS súradnicami sa používa Haversine vzorec, ktorý zohľadňuje zakrivenie Zeme. Výsledkom je vzdialenosť v kilometroch (polomer Zeme $R = 6371$ km):

```
1  private fun haversineKm(
2      lat1: Double, lon1: Double,
3      lat2: Double, lon2: Double
4  ): Double {
5      val dLat = Math.toRadians(lat2 - lat1)
6      val dLon = Math.toRadians(lon2 - lon1)
7      val a = sin(dLat / 2).pow(2) +
8              cos(Math.toRadians(lat1)) *
9              cos(Math.toRadians(lat2)) *
10             sin(dLon / 2).pow(2)
11     return 6371.0 * 2 * asin(sqrt(a))
12 }
```

Výpis kódu 17: Haversine vzorec

Filter bottom sheet — UI: Filter sa zobrazí ako `ModalBottomSheet` po kliknutí na tlačidlo „Nájsť udalosť". Používateľ môže filtrovať podľa:

- názvu — fulltextové vyhľadávanie (case-insensitive),
- kategórie a podkategórie — výber z `CATEGORY_MAP`,
- počtu osôb — slider v rozsahu 0–200,
- maximálnej ceny — slider 0–150 €; hodnota nad 150 € znamená bez obmedzenia,
- vzdialenosti — slider 1–50 km od GPS polohy používateľa; hodnota nad 50 km znamená kdekoľvek,
- dátumu začiatku — výber cez `DatePickerDialog` a `TimePickerDialog`.

Po kliknutí na „Použiť"sa zostavia kritériá a filtrujú sa markery na mape:

```
val criteria = FilterCriteria(
    name          = name.trim().ifEmpty { null },
    category      = selectedCategory.ifEmpty { null },
    maxPrice      = if (maxPrice >= 150f) Double.MAX_VALUE
                    else maxPrice.toDouble(),
    maxDistanceKm = if (maxDistance < 50f) maxDistance.toDouble()
                    else null,
    refLat        = userLocation?.latitude  ?: 0.0,
    refLng        = userLocation?.longitude ?: 0.0
)
val filtered = EventFilterService(criteria).filter(events)
markers.values.forEach { it.isVisible = false }
filtered.forEach { event -> markers[event.id]?.isVisible = true }
```

Výpis kódu 18: Aplikovanie filtrov na mapu

Výsledok filtra sa prejaví priamo na mape — zobrazia sa len markery eventov spĺňajúcich všetky zadané kritériá. Stav filtra sa ukladá do `AppViewModel.filterState`, čím sa zachováva pri navigácii medzi obrazovkami.

4.3.5 Hodnotenie udalostí

Systém hodnotenia umožňuje účastníkom ohodnotiť udalosť po jej skončení. Hodnotenie je implementované priamo v `AppViewModel`.

Dátový model: Hodnotenie je uložené priamo v dokumente eventu v troch poliach:

```
val rating      : Double       = 0.0          // vážený priemer
val totalRatings: Int          = 0            // počet hodnotení
val ratedBy     : List<String> = emptyList() // UIDs ktorí už hodnotili
```

Výpis kódu 19: Polia hodnotenia v dátovom modeli eventu

Logika hodnotenia: Nový priemer sa vypočíta ako vážený priemer — starý priemer sa vynásobí počtom predchádzajúcich hodnotení, pripočíta sa nové hodnotenie a výsledok sa vydelí novým celkovým počtom. Týmto spôsobom sa zachová presnosť bez nutnosti ukladať každé individuálne hodnotenie:

```
fun rateEvent(eventId: String, userId: String, stars: Int) {
    val event = _events[eventId] ?: return
    if (userId in event.ratedBy) return  // každý môže hodnotiť len raz

    val newTotal  = event.totalRatings + 1
    val newRating = ((event.rating * event.totalRatings) + stars) / newTotal

    val updated = event.copy(
        rating       = newRating,
        totalRatings = newTotal,
        ratedBy      = event.ratedBy + userId
    )
    _events[eventId] = updated
    selectedEvent = updated
    viewModelScope.launch {
        db.collection("events").document(eventId).update(mapOf(
            "rating"       to newRating,
            "totalRatings" to newTotal,
            "ratedBy"      to updated.ratedBy
        )).await()
    }
}
```

Výpis kódu 20: Implementácia metódy rateEvent

Pravidlá hodnotenia:

- Hodnotiť môžu len účastníci eventu — nie tvorca ani cudzí používatelia.
- Každý používateľ môže hodnotiť len raz — kontrola prebieha cez zoznam `ratedBy`.
- Hodnotenie je v rozsahu 1 až 5 hviezd.

- Priemerné hodnotenie vstupuje ako jeden z faktorov do odporúčacieho systému s váhou 16 %.

Používateľské rozhranie: Sekcia hodnotenia sa v `EventDetailScreen` (Tab 0) zobrazí len účastníkovi, ktorý ešte nehodnotil:

```
val isAttending = currentUserId in event.attendees
val hasRated    = currentUserId in event.ratedBy

// zobrazí sa len účastníkovi ktorý ešte nehodnotil
if (isAttending && !isCreator && !hasRated) {
// 5 hviezd - Row s ikonami
}
```

Výpis kódu 21: Podmienka zobrazenia sekcie hodnotenia

Aktuálne priemerné hodnotenie a celkový počet hodnotení sú viditeľné pre všetkých používateľov.

4.3.6 Check-in systém

Check-in systém umožňuje organizátorovi overiť fyzickú prítomnosť účastníkov na udalosti. Pri tvorbe eventu si organizátor zvolí jeden zo štyroch typov check-inu, ktoré sú zhrnuté v tabuľke 6.

Tabuľka 6: Typy check-inu

Hodnota	Popis
`none`	Check-in sa nevyžaduje
`qr`	Účastník naskenuje QR kód
`manual`	Organizátor manuálne označí prítomnosť
`qr_and_manual`	Kombinácia oboch predchádzajúcich

Generovanie QR kódu: QR kód sa generuje pomocou knižnice ZXing priamo na zariadení organizátora (`QrCodeScreen.kt`). Obsahom QR kódu je `eventId` — unikátny identifikátor udalosti z Firestore:

```
private fun generateQrBitmap(content: String, size: Int = 512): Bitmap? {
    val bitMatrix = QRCodeWriter()
        .encode(content, BarcodeFormat.QR_CODE, size, size)
    val bitmap = Bitmap.createBitmap(size, size, Bitmap.Config.RGB_565)
    for (x in 0 until size) {
        for (y in 0 until size) {
```

```
            bitmap.setPixel(x, y,
                if (bitMatrix[x, y]) Color.BLACK else Color.WHITE
            )
        }
    }
    return bitmap
}
```

Výpis kódu 22: Generovanie QR kódu pomocou ZXing

Skenovanie QR kódu: Skenovanie je implementované v `QrScanScreen.kt` pomocou CameraX na zobrazenie náhľadu kamery a ML Kit Barcode Scanning na rozpoznanie kódu. Obrazovka najprv požiada o oprávnenie `CAMERA`:

```
val barcodeScanner = BarcodeScanning.getClient(
    BarcodeScannerOptions.Builder()
        .setBarcodeFormats(Barcode.FORMAT_QR_CODE)
        .build()
)

val analysis = ImageAnalysis.Builder()
    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
    .build()

analysis.setAnalyzer(ContextCompat.getMainExecutor(ctx)) { imageProxy ->
    val image = InputImage.fromMediaImage(
        mediaImage, imageProxy.imageInfo.rotationDegrees
    )
    barcodeScanner.process(image)
        .addOnSuccessListener { barcodes ->
            barcodes.firstOrNull()?.rawValue?.let { value ->
                scanned = true
                onScanResult(value)  // value = eventId
            }
        }
}
```

Výpis kódu 23: Skenovanie QR kódu pomocou ML Kit

Stratégia `STRATEGY_KEEP_ONLY_LATEST` zabezpečuje, že sa vždy spracuje len posledný zachytený snímok — staršie sa zahodia, čím sa predchádza preťaženiu. Flag `scanned` zabraňuje viacnásobnému spracovaniu toho istého QR kódu. Po úspešnom naskenovaní sa zavolá `appViewModel.checkInAttendee(eventId, userId)`.

Check-in logika: Metóda `checkInAttendee()` overí, či je používateľ účastníkom eventu a či ešte nebol check-inovaný:

```
fun checkInAttendee(eventId: String, userId: String) {
    val event = _events[eventId] ?: return
    if (userId !in event.attendees) {
        checkInError = "Nie si účastníkom tejto udalosti."
        return
    }
    if (userId in event.checkedIn) {
        checkInError = "Už si zaregistrovaný."
        return
    }
    val updated = event.copy(checkedIn = event.checkedIn + userId)
    _events[eventId] = updated
    viewModelScope.launch {
        db.collection("events").document(eventId)
            .update("checkedIn", updated.checkedIn).await()
        checkInSuccess = true
    }
}
```

Výpis kódu 24: Logika check-inu v AppViewModel

Organizátor môže check-in aj zrušiť volaním `checkOutAttendee()`, ktorá odstráni `userId` zo zoznamu `checkedIn`.

Manuálny check-in: Pri type `manual` alebo `qr_and_manual` vidí organizátor v Tab Účastníci pri každom účastníkovi prepínač ○ / ✓. Kliknutím sa zavolá `checkInAttendee()` alebo `checkOutAttendee()`. Účastník s potvrdenou prítomnosťou má zelenú odznak „✓ Prítomný".

Väzba na penalizačný systém: Ak je `checkInType` ≠ `none` a účastník sa neodhlási ani nezaregistruje check-in, po skončení eventu mu je automaticky zaznamenaný no-show. Tento mechanizmus je podrobnejšie popísaný v nasledujúcej podkapitole.

4.3.7 Systém penalizácií (No-show)

Penalizačný systém motivuje používateľov dodržiavať záväzky voči udalostiam. Ak sa používateľ prihlási na udalosť s povoleným check-inom, fyzicky sa nezúčastní a ani sa včas neodhlási, systém mu zaznamená no-show.

Dátový model: Penalizácie sú uložené priamo v profile používateľa (`UserProfile`):

```
val noShowTimestamps: List<Long>   = emptyList() // čas každého no-show (epoch ms)
val noShowEventIds  : List<String> = emptyList() // dedup - eventy kde bol no-show

val banned          : Boolean      = false        // true = účet zablokovaný
```

Výpis kódu 25: Polia no-show v UserProfile

Pomocná metóda počíta no-show len za posledných 30 dní:

```
fun recentNoShowCount(): Int {
    val cutoff = System.currentTimeMillis() - 30L * 24 * 3_600_000
    return noShowTimestamps.count { it > cutoff }
}
```

Výpis kódu 26: Výpočet počtu no-show za posledných 30 dní

Dva spôsoby vzniku no-show:

1. **Neskoré odhlásenie** — používateľ sa odhlási po uplynutí `cancelDeadlineHours`:

```
fun leaveEvent(eventId: String, userId: String) {
    val applyPenalty = event.isPastCancelDeadline()
    // ... odhlásenie z eventu ...
    if (applyPenalty) recordNoShow(userId, eventId)
}
```

2. **Neúčasť** — systém pri štarte skontroluje všetky minulé eventy, kde bol používateľ prihlásený, ale nebol check-inovaný:

```
fun checkNoShows() {
    for (event in _events.values) {
        if (event.checkInType == "none") continue   // check-in sa nevyžadoval
        if (uid !in event.attendees)    continue    // nebol prihlásený
        if (uid in event.checkedIn)     continue    // bol prítomný
        if (event.id in profile.noShowEventIds) continue  // dedup
        val eventEnd = event.dateTo?.time
            ?: event.dateFrom?.time ?: continue
        if (eventEnd > now) continue                // ešte neskončil
        recordNoShow(uid, event.id)
    }
}
```

Metóda `checkNoShows()` sa volá automaticky po každom prihlásení v sekvencii `loadUserProfile()` → `checkNoShows()`.

Zaznamenanie no-show: Metóda `recordNoShow()` aktualizuje profil používateľa a v prípade dosiahnutia limitu nastaví príznak zablokovania:

```
suspend fun recordNoShow(userId: String, eventId: String) {
    val now           = System.currentTimeMillis()
```

```
    val newTimestamps = profile.noShowTimestamps + now
    val newEventIds   = profile.noShowEventIds   + eventId
    val recentCount   = newTimestamps.count {
        it > now - 30L * 24 * 3_600_000
    }
    val shouldBan = recentCount >= 3

    val updates = mutableMapOf<String, Any>(
        "noShowTimestamps" to newTimestamps,
        "noShowEventIds"   to newEventIds
    )
    if (shouldBan) updates["banned"] = true

    db.collection("users").document(userId).update(updates).await()
}
```

Výpis kódu 27: Implementácia recordNoShow

Stupňovité varovania: Po zaznamenaní no-show sa používateľovi zobrazí upozornenie podľa aktuálneho počtu, ako je uvedené v tabuľke 7.

Tabuľka 7: Správanie systému podľa počtu no-show za posledných 30 dní

Počet no-show	Akcia
1	Toast: „Máš 1/3 neúčastí za posledných 30 dní."
2	Toast: „Máš 2/3! Ďalšia ťa zablokuje!"
3	`banned = true` → odhlásenie → „Váš účet bol zablokovaný."

Zablokovanie účtu: Ak `shouldBan == true`, pole `banned` sa nastaví na `true` v databáze Firestore. Pri ďalšom prihlásení `loadUserProfile()` toto pole skontroluje a okamžite odhlási používateľa:

```
if (profile.banned) {
    FirebaseAuth.getInstance().signOut()
    loginBannedError = true
    return@launch
}
```

Výpis kódu 28: Kontrola zablokovania pri prihlásení

No-show sa počítajú výlučne za posledných 30 dní — po uplynutí tohto obdobia sa staré záznamy nezapočítavajú, čo umožňuje rehabilitáciu používateľa bez nutnosti manuálneho zásahu administrátora.

4.3.8 Systém nahlasovania

Systém nahlasovania umožňuje používateľom upozorniť na nevhodný obsah — či už udalosť alebo iného používateľa. Pri dosiahnutí stanoveného počtu nahlásení sa obsah automaticky odstráni bez nutnosti zásahu administrátora.

Dátový model: Každé nahlásenie je reprezentované triedou `Report` a uložené do Firestore kolekcie `reports`:

```
data class Report(
    val id        : String = "",
    val reporterId: String = "",  // kto nahlásil
    val targetId  : String = "",  // ID udalosti alebo používateľa
    val targetType: String = "",  // "event" | "user"
    val reason    : String = "",
    val timestamp : Long   = System.currentTimeMillis()
)
```

Výpis kódu 29: Dátová trieda Report

Dôvody nahlasovania: Zoznam dôvodov a hranica automatickej akcie sú definované ako konštanty v `AppViewModel`:

```
companion object {
    val REPORT_REASONS = listOf(
        "Nevhodný obsah",
        "Spam",
        "Falošná udalosť / profil",
        "Obťažovanie",
        "Iné"
    )
    private const val REPORTS_TO_BAN = 3
}
```

Výpis kódu 30: Konštanty systému nahlasovania

Používateľské rozhranie: Nahlásenie sa vyvolá kliknutím na ikonu príznaku v `EventDetailScreen` (pre udalosť) alebo v `PublicProfileScreen` (pre cudzí profil). Zobrazí sa `AlertDialog` so zoznamom dôvodov ako prepínače (*radio buttons*). Tlačidlo „Nahlásiť"je aktívne len ak je niektorý dôvod vybratý.

Logika nahlasovania udalosti: Metóda `reportEvent()` prebieha v troch krokoch — kontrola duplikátu, uloženie nahlásenia a prípadné automatické vymazanie udalosti:

```
fun reportEvent(reporterId: String, eventId: String, reason: String) {
    // 1. Kontrola duplikátu - jeden používateľ môže nahlásiť len raz
    val dup = db.collection("reports")
        .whereEqualTo("reporterId", reporterId)
        .whereEqualTo("targetId",   eventId)
        .whereEqualTo("targetType", "event")
        .get().await()
    if (!dup.isEmpty) {
        reportError = "Túto udalosť si už nahlásil/a."
        return
    }

    // 2. Uloženie nahlásenia
    val docRef = db.collection("reports").document()
    docRef.set(Report(...).toMap()).await()

    // 3. Ak celkový počet nahlásení >= 3, udalosť sa automaticky vymaže
    val total = db.collection("reports")
        .whereEqualTo("targetId",   eventId)
        .whereEqualTo("targetType", "event")
        .get().await()
    if (total.size() >= REPORTS_TO_BAN) deleteEvent(eventId)
}
```

Výpis kódu 31: Implementácia metódy reportEvent

Logika nahlasovania používateľa: Metóda `reportUser()` funguje rovnakým spôsobom. Pri dosiahnutí prahu sa namiesto mazania udalosti nastaví príznak zablokovania na profile používateľa:

```
if (total.size() >= REPORTS_TO_BAN) {
    db.collection("users")
      .document(targetUserId)
      .update("banned", true)
      .await()
}
```

Výpis kódu 32: Automatické zablokovanie používateľa

Zablokovaný používateľ je pri ďalšom prihlásení automaticky odhlásený — rovnaký mechanizmus ako pri penalizačnom systéme (viď predchádzajúcu podkapitolu).

Pravidlá systému: Tabuľka 8 sumarizuje pravidlá platné pre systém nahlasovania.

Tabuľka 8: Pravidlá systému nahlasovania

Pravidlo	Detail
Kto môže nahlásiť	Každý prihlásený používateľ okrem tvorcu udalosti a seba samého.
Duplikát	Jeden používateľ môže nahlásiť ten istý cieľ len raz.
Prah	3 unikátne nahlásenia spustia automatickú akciu.
Udalosť	Vymazaná z Firestore aj z in-memory mapy.
Používateľ	`banned = true` → odhlásenie pri ďalšom prihlásení.

4.3.9 Systém priateľstiev

Systém priateľstiev umožňuje používateľom nadviazať spojenie a komunikovať priamo cez správy. Implementácia pokrýva posielanie žiadostí, ich prijímanie a odmietanie, odstraňovanie priateľov a priamu komunikáciu.

Dátový model: Každé priateľstvo je reprezentované triedou `Friendship` a uložené do Firestore kolekcie `friendships`:

```
data class Friendship(
    val id          : String = "",
    val requesterId: String = "",        // kto poslal žiadosť
    val receiverId : String = "",        // kto prijíma žiadosť
    val status      : String = "pending", // "pending" | "accepted"
    val timestamp   : Long   = 0L
)
```

Výpis kódu 33: Dátová trieda Friendship

Stavy priateľstva: Pre každú dvojicu používateľov môže existovať práve jeden vzťah. Metóda `getFriendshipStatus()` vráti jeden zo štyroch stavov uvedených v tabuľke 9:

```
fun getFriendshipStatus(currentUserId: String, otherUserId: String): String {
    val f = _friendships.firstOrNull {
        (it.requesterId == currentUserId && it.receiverId == otherUserId) ||
        (it.requesterId == otherUserId   && it.receiverId == currentUserId)
    } ?: return "none"

    return when {
        f.status == "accepted"         -> "friends"
```

```
        f.requesterId == currentUserId -> "pending_sent"
        else                           -> "pending_received"
    }
}
```

Výpis kódu 34: Určenie stavu priateľstva

Tabuľka 9: Stavy priateľstva

Stav	Význam
`none`	Žiadny vzťah medzi používateľmi.
`pending_sent`	Aktuálny používateľ poslal žiadosť.
`pending_received`	Aktuálny používateľ dostal žiadosť.
`friends`	Priateľstvo je akceptované.

Stav sa využíva v `PublicProfileScreen` na zobrazenie správneho tlačidla — „Pridať priateľa", „Žiadosť odoslaná", „Prijať / Odmietnuťalebo „Odstrániť priateľa".

Načítanie priateľstiev: Pri prihlásení sa načítajú všetky dokumenty, kde je používateľ buď odosielateľom alebo prijímateľom. Odmietnuté žiadosti sa nenačítavajú — zabraňuje sa tým opakovanému posielaniu žiadostí od toho istého používateľa:

```
fun loadFriendships(uid: String) {
    val sent     = db.collection("friendships")
        .whereEqualTo("requesterId", uid).get().await()
    val received = db.collection("friendships")
        .whereEqualTo("receiverId",  uid).get().await()
    (sent.documents + received.documents).forEach { doc ->
        val f = Friendship.fromFirestore(doc.id, doc.data!!)
        if (f.status != "declined") _friendships.add(f)
    }
}
```

Výpis kódu 35: Načítanie priateľstiev z Firestore

Odoslanie žiadosti: Pred vytvorením novej žiadosti sa skontroluje, či medzi dvojicou používateľov už vzťah existuje:

```
fun sendFriendRequest(fromId: String, toId: String) {
    val alreadyExists = _friendships.any {
        (it.requesterId == fromId && it.receiverId == toId) ||
        (it.requesterId == toId   && it.receiverId == fromId)
    }
    if (alreadyExists) {
```

```
        friendshipError = "Žiadosť o priateľstvo už existuje."
        return
    }
    val friendship = Friendship(
        id          = docRef.id,
        requesterId = fromId,
        receiverId  = toId,
        status      = "pending",
        timestamp   = System.currentTimeMillis()
    )
    docRef.set(friendship.toMap()).await()
    _friendships.add(friendship)
}
```

Výpis kódu 36: Odoslanie žiadosti o priateľstvo

Odpoveď na žiadosť: Pri prijatí sa stav aktualizuje na `accepted`; pri odmietnutí sa dokument z Firestore aj z lokálnej pamäte vymaže:

```
fun respondToFriendRequest(friendshipId: String, accept: Boolean) {
    if (accept) {
        db.collection("friendships").document(friendshipId)
            .update("status", "accepted").await()
        _friendships[idx] = _friendships[idx].copy(status = "accepted")
    } else {
        db.collection("friendships").document(friendshipId)
            .delete().await()
        _friendships.removeAll { it.id == friendshipId }
    }
}
```

Výpis kódu 37: Prijatie alebo odmietnutie žiadosti

Odstránenie priateľa: Metóda `removeFriend()` vyhľadá akceptované priateľstvo medzi dvojicou a vymaže ho z Firestore aj z lokálnej pamäte:

```
fun removeFriend(currentUserId: String, otherUserId: String) {
    val f = _friendships.firstOrNull {
        it.status == "accepted" &&
        ((it.requesterId == currentUserId && it.receiverId == otherUserId) ||
         (it.requesterId == otherUserId   && it.receiverId == currentUserId))
    } ?: return
    _friendships.remove(f)
    db.collection("friendships").document(f.id).delete().await()
}
```

Výpis kódu 38: Odstránenie priateľa

Priama komunikácia: Priatelia si môžu písať správy cez `FriendChatScreen`. ID konverzácie sa generuje deterministicky zo zoradených UID oboch používateľov — zaručuje sa tak, že obaja vždy pristupujú k tomu istému dokumentu bez ohľadu na to, kto rozhovor začal:

```
private fun directConversationId(uid1: String, uid2: String) =
    if (uid1 < uid2) "${uid1}_${uid2}" else "${uid2}_${uid1}"
```

Výpis kódu 39: Deterministické generovanie ID konverzácie

Správy sa ukladajú do Firestore kolekcie `directMessages/{conversationId}/messages` a synchronizujú sa v reálnom čase cez `snapshotListener` — rovnaký mechanizmus ako chat v udalostiach.

Používateľské rozhranie: Obrazovka `FriendsScreen` je dostupná z bočného menu pod položkou „Priateliaä je rozdelená do dvoch sekcií:

- **Žiadosti o priateľstvo** — zobrazujú sa len ak existujú, s tlačidlami „Prijaťä „Odmietnuť".
- **Priatelia** — zoznam akceptovaných priateľov s možnosťou otvoriť priamu konverzáciu alebo priateľa odstrániť.

4.4 Implementácia systému pozvánok

Systém pozvánok umožňuje organizátorovi udalosti prizvať konkrétnych používateľov priamo cez ich e-mailovú adresu. Pozvaný používateľ dostane pozvánku v zozname *Pozvánky*, kde ju môže prijať alebo odmietnuť.

Dátový model: Každá pozvánka je reprezentovaná objektom `Invitation` a uložená ako dokument vo Firestore kolekcii `invitations`. Model uchováva identifikátory odosielateľa aj príjemcu, referenciu na udalosť a aktuálny stav pozvánky:

```
data class Invitation(
    val id           : String = "",
    val eventId      : String = "",
    val eventTitle   : String = "",
    val eventPlace   : String = "",
    val fromUserId   : String = "",
    val fromUserName : String = "",
    val toUserId     : String = "",
    val toEmail      : String = "",
    val status       : String = "pending", // "pending" | "accepted" | "declined"
    val createdAt    : Date   = Date()
)
```

Výpis kódu 40: Dátový model Invitation

Odoslanie pozvánky: Organizátor môže odoslať pozvánku priamo z detailu udalosti kliknutím na ikonu `PersonAdd` v hornej lište. Zobrazí sa dialóg s textovým poľom pre zadanie e-mailovej adresy pozývaného používateľa.

Logika odosielania je implementovaná v metóde `sendInvitation` v `AppViewModel`. Metóda najprv vyhľadá používateľa podľa e-mailovej adresy v kolekcii `users`. Ak používateľ neexistuje, nastaví sa chybová správa. Následne sa overí, či pre danú kombináciu `eventId` a `toUserId` pozvánka už existuje, čím sa zabraňuje duplicitným pozvánkam:

```
fun sendInvitation(
    event        : Event,
    fromUserId   : String,
    fromUserName : String,
    toEmail      : String
) {
    viewModelScope.launch {
        // Vyhladanie prijemcu podla e-mailu
        val userSnap = db.collection("users")
            .whereEqualTo("email", toEmail).get().await()
        if (userSnap.isEmpty) {
            invitationError = "Pouzivatel s tymto e-mailom neexistuje."
            return@launch
        }
        val toUserId = userSnap.documents.first().id

        // Kontrola duplicity
        val existing = db.collection("invitations")
            .whereEqualTo("eventId",   event.id)
            .whereEqualTo("toUserId",  toUserId)
            .get().await()
        if (!existing.isEmpty) {
            invitationError = "Tento pouzivatel uz bol pozvany."
            return@launch
        }

        val docRef     = db.collection("invitations").document()
        val invitation = Invitation(
            id           = docRef.id,
            eventId      = event.id,
            eventTitle   = event.title,
            eventPlace   = event.place,
            fromUserId   = fromUserId,
```

```
            fromUserName = fromUserName,
            toUserId     = toUserId,
            toEmail      = toEmail
        )
        docRef.set(invitation.toMap()).await()
        invitationSuccess = "Pozvanka bola odoslana."
    }
}
```

Výpis kódu 41: Odoslanie pozvánky s kontrolou duplicity

Načítanie a zobrazenie pozvánok: Pozvánky sa načítavajú pri spustení aplikácie v metóde `loadInvitations`, ktorá z Firestore načíta všetky dokumenty v kolekcii `invitations`, kde `toUserId` zodpovedá aktuálne prihlásenému používateľovi.

Obrazovka `MyInvitationsScreen` rozdeľuje pozvánky do dvoch skupín: *Čakajúce* (stav `pending`) a *Vybavené* (stav `accepted` alebo `declined`). Každá pozvánka je zobrazená ako karta s názvom udalosti, miestom konania, menom odosielateľa a farebným stavovým odznačkom.

Reakcia na pozvánku: Prijatie alebo odmietnutie pozvánky rieši metóda `respondToInvitation`. V prípade prijatia sa stav aktualizuje na `accepted` a zároveň sa automaticky zavolá `joinEvent`, čím sa používateľ priradí medzi účastníkov udalosti. Pri odmietnutí sa stav nastaví iba na `declined` bez ďalšej akcie:

```
fun respondToInvitation(
    invitation    : Invitation,
    accept        : Boolean,
    currentUserId : String
) {
    val newStatus = if (accept) "accepted" else "declined"
    val idx = _invitations.indexOfFirst { it.id == invitation.id }
    if (idx >= 0) _invitations[idx] = invitation.copy(status = newStatus)

    viewModelScope.launch {
        db.collection("invitations").document(invitation.id)
            .update("status", newStatus).await()
        if (accept) joinEvent(invitation.eventId, currentUserId)
    }
}
```

Výpis kódu 42: Reakcia na pozvánku

Výsledok operácie je používateľovi oznámený prostredníctvom správy `Snackbar`, ktorú riadia stavové premenné `invitationSuccess` a `invitationError` v `AppViewModel`.

4.5 Implementácia systému lokálnych notifikácií

Aplikácia Joinly informuje používateľov o nadchádzajúcich udalostiach prostredníctvom lokálnych notifikácií systému Android. Na rozdiel od push notifikácií, ktoré vyžadujú serverovú infraštruktúru, lokálne notifikácie sú naplánované priamo na zariadení a nevyžadujú aktívne internetové pripojenie v momente doručenia.

Architektúra systému: Systém pozostáva z troch spolupracujúcich komponentov:

- **NotificationHelper** — singleton objekt zodpovedný za plánovanie a rušenie notifikácií,
- **NotificationReceiver** — prijímač systémových správ, ktorý zostavuje a zobrazuje notifikáciu,
- **Notification** — dátový model zabezpečujúci perzistenciu naplánovaných notifikácií vo Firestore.

Notifikačný kanál a správa oprávnení: Od verzie Android 8.0 (Oreo) je povinné zaregistrovať notifikačný kanál pred prvým použitím. Kanál s identifikátorom `joinly_events` a prioritou `IMPORTANCE_HIGH` sa vytvára pri štarte aplikácie v `LaunchedEffect` v kompozícii `App.kt`:

```
fun createChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID, CHANNEL_NAME, NotificationManager.IMPORTANCE_HIGH
        ).apply {
            description = "Upozornenia na nadchadzajuce udalosti"
        }
        context.getSystemService(NotificationManager::class.java)
            .createNotificationChannel(channel)
    }
}
```

Výpis kódu 43: Vytvorenie notifikačného kanála

Od verzie Android 13 (Tiramisu) vyžaduje zobrazovanie notifikácií explicitné oprávnenie `POST_NOTIFICATIONS`. Aplikácia ho vyžiada pri prvom spustení pomocou `rememberLauncherForActivityResult`. Oprávnenie `SCHEDULE_EXACT_ALARM` je deklarované v súbore `AndroidManifest.xml` a umožňuje presné načasovanie budíkov.

Plánovanie notifikácie: Notifikácia sa naplánuje automaticky v momente, keď sa používateľ pripojí k udalosti ako potvrdený účastník (nie ako čakajúci v poradovníku). Upozornenie sa odošle hodinu pred začiatkom udalosti. Plánovanie zabezpečuje `AlarmManager` prostredníctvom metódy `setExactAndAllowWhileIdle`, ktorá garantuje spustenie aj v prípade, že je zariadenie v úspornom režime (*Doze mode*). Na zariadeniach s Android 12+ sa pred použitím presného alarmu overí dostupnosť oprávnenia:

```
fun scheduleEventNotification(context: Context, event: Event) {
    val dateFrom = event.dateFrom ?: return
    val notifyAt = dateFrom.time - NOTIFY_BEFORE_MS   // 1 hodina pred
    if (notifyAt <= System.currentTimeMillis()) return

    val alarmManager = context.getSystemService(AlarmManager::class.java)
    val pending = buildPendingIntent(context, event)

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S
            && !alarmManager.canScheduleExactAlarms()) {
        alarmManager.setAndAllowWhileIdle(
            AlarmManager.RTC_WAKEUP, notifyAt, pending
        )
    } else {
        alarmManager.setExactAndAllowWhileIdle(
            AlarmManager.RTC_WAKEUP, notifyAt, pending
        )
    }
}
```

Výpis kódu 44: Plánovanie notifikácie pomocou AlarmManager

Každý alarm je identifikovaný pomocou `PendingIntent`, do ktorého sú vložené metadáta udalosti — `eventId`, `eventTitle`, `eventPlace` a čas začiatku. Ako kľúč pre `requestCode` sa používa `eventId.hashCode()`, čo zaručuje jednoznačnú identifikáciu každej naplánovanej notifikácie.

Zobrazenie notifikácie: Keď `AlarmManager` spustí alarm, systém doručí broadcast triede `NotificationReceiver`, ktorá implementuje `BroadcastReceiver`. Prijímač z `Intent` extrahuje metadáta udalosti, zostaví text správy a zobrazí notifikáciu cez `NotificationManagerCompat`:

```
override fun onReceive(context: Context, intent: Intent) {
    val eventTitle = intent.getStringExtra("eventTitle") ?: "Udalost"
    val eventPlace = intent.getStringExtra("eventPlace") ?: ""
    val eventTime  = intent.getLongExtra("eventTime", 0L)

    val timeStr = SimpleDateFormat("HH:mm", Locale.getDefault())
```

```
7            .format(Date(eventTime))
8        val body = "\"$eventTitle\" zacina o $timeStr - $eventPlace"
9
10       val notification = NotificationCompat.Builder(
11               context, NotificationHelper.CHANNEL_ID
12           )
13           .setSmallIcon(R.drawable.ic_star)
14           .setContentTitle("Joinly -- Event o hodinu!")
15           .setContentText(body)
16           .setStyle(NotificationCompat.BigTextStyle().bigText(body))
17           .setPriority(NotificationCompat.PRIORITY_HIGH)
18           .setAutoCancel(true)
19           .build()
20
21       NotificationManagerCompat.from(context)
22           .notify(eventId.hashCode(), notification)
23   }
```

Výpis kódu 45: Zostavenie a zobrazenie notifikácie

Trieda `NotificationReceiver` je registrovaná v súbore `AndroidManifest.xml` ako prijímač systémových správ.

Zrušenie notifikácie: Ak používateľ opustí udalosť alebo ak organizátor udalosť vymaže, naplánovaný alarm sa zruší volaním `cancelEventNotification`. Zrušenie sa vykoná vyhľadaním existujúceho `PendingIntent` s príznakmi `FLAG_NO_CREATE` — ak alarm neexistuje, metóda nevyvolá chybu:

```
1  fun cancelEventNotification(context: Context, eventId: String) {
2      val alarmManager = context.getSystemService(AlarmManager::class.java)
3      val intent = Intent(context, NotificationReceiver::class.java)
4      val pending = PendingIntent.getBroadcast(
5          context,
6          eventId.hashCode(),
7          intent,
8          PendingIntent.FLAG_NO_CREATE or PendingIntent.FLAG_IMMUTABLE
9      )
10     pending?.let { alarmManager.cancel(it) }
11 }
```

Výpis kódu 46: Zrušenie naplánovanej notifikácie

Perzistencia vo Firestore: Okrem lokálneho alarmu sa každá naplánovaná notifikácia uloží aj do Firestore kolekcie `notifications`. Dátový model `Notification` uchováva `userId`, `eventId`, `eventTitle`, čas odoslania `scheduledAt` a príznak `sent`.

Tento záznam sa pri opustení udalosti alebo vymazaní eventu tiež odstráni. Perzistencia vo Firestore slúži ako záloha pre prípad, keby sa lokálny alarm stratil (napríklad po reštarte zariadenia).

Záver

„V tejto príručke sme sa snažili poskytnúť praktické pokyny pre vypracovanie záverečných prác na Fakulte elektrotechniky a informatiky STU v Bratislave. Obsahuje dôležité informácie o formálnej stránke dokumentu, ktoré sú nevyhnutné pre zabezpečenie jasnej štruktúry a správneho formátovania práce. Naša príručka kladie dôraz na dodržiavanie základných pravidiel, ktoré pomáhajú minimalizovať chyby a zlepšujú celkovú kvalitu predkladaných dokumentov.

Rovnakú pozornosť venujeme aspektom, ako sú formátovanie textu, zarovnávanie, riadkovanie a použitie preddefinovaných štýlov, ktoré prispievajú k profesionálnemu vzhľadu dokumentu. Dôkladné spracovanie matematických rovníc a grafov je ďalším dôležitým prvkom, ktorý môže ovplyvniť hodnotenie práce. Práca s týmito aspektmi zaisťuje, že obsah je prehľadný a presne odráža zamýšľané myšlienky.

V neposlednom rade sa naša príručka zameriava na význam citovania externých zdrojov, čo je zásadné pre zachovanie integrity a etických štandardov pri písaní akademických textov. Správne citovanie nielenže posilňuje argumentáciu, ale tiež pomáha predchádzať plagiátorstvu, ktoré môže mať vážne následky na akademickú kariéru.

Dodržiavanie týchto pokynov umožňuje vyhnúť sa bežným chybám a zvyšuje kvalitu záverečných prác, čím sa zlepšuje celkový dojem z akademických schopností. Dobre vypracovaná práca môže viesť k pozitívnemu hodnoteniu a uznaniu zo strany hodnotiteľov, čo je cieľom každého študenta.

Celkovo je príručka navrhnutá ako cenný nástroj na podporu pri písaní záverečných prác. Poskytnuté informácie a odporúčania prispejú k dosiahnutiu akademických cieľov a pomôžu vypracovať kvalitné a odborné dokumenty, ktoré budú spĺňať štandardy fakulty.“

(ChatGPT 3, 2024)

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Worde. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, neprípustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

Použitie nástrojov umelej inteligencie

OpenAI (2025), ChatGPT 4o, časť **??**, generovanie vzorca záznamu použitia AI.

OpenAI (2024), ChatGPT 3, Záver, generovanie textu.

OpenAI (2025), ChatGPT 4o, dodatok B, generovanie kódu programu.

OpenAI (2024), ChatGPT 3, dodatok C, generovanie zoznamu.

Dodatok A: Algoritmus

Algoritmus 1 Vypočítaj $y = x^n$

Require: $n \geq 0 \vee x \neq 0$

Ensure: $y = x^n$

```
y ⇐ 1
if n < 0 then
    X ⇐ 1/x
    N ⇐ −n
else
    X ⇐ x
    N ⇐ n
end if
while N ≠ 0 do
    if N is even then
        X ⇐ X × X
        N ⇐ N/2
    else {N is odd}
        y ⇐ y × X
        N ⇐ N − 1
    end if
end while
```

Dodatok B: Výpis dlhého kódu

Dlhšie kódy, prípadne kompletné programy vypisujeme do dodatkov práce. Nasledujúci program v jazyku Python vygeneruje súbor `thesis.tex`, ktorý je funkčne totožný s hlavným súborom tohto dokumentu. Stačí vyplniť prehľadný formulár pri volaní funkcie `generate_thesis_tex` na konci kódu a získate hlavný súbor vašej práce.

```
def generate_thesis_tex(
    author,
    reg_nr,
    title,
    title_en,
    day,
    month,
    year,
    keywords,
    keywords_en,
    study_programme,
    study_programme_en,
    study_field,
    study_field_en,
    training_workplace,
    training_workplace_en,
    supervisor,
    consultant=None,
    thesis_type="bp",
    language="sk",
):

    tex_class_option = thesis_type if thesis_type in ["bp", "dp"] else "bp"
    tex_language_option = "en" if language == "en" else ""

    if tex_language_option:
        document_class = f"\\documentclass[{tex_class_option},{tex_language_option
    }]{{FEIstyle}}"
    else:
        document_class = f"\\documentclass[{tex_class_option}]{{FEIstyle}}"

    tex_content = f"""
{document_class}

% Author of the thesis
\\FEIauthor{{{author}}}
```

```
% Evidence number
\\FEIregNr{{{reg_nr}}}

% Title
\\FEItitle{{{title}}}
\\FEItitleEn{{{title_en}}}

% Date
\\FEIdate{{{day}}}{{{month}}}{{{year}}}

% Keywords
\\FEIkeywords{{{keywords}}}
\\FEIkeywordsEn{{{keywords_en}}}

% Further details
\\FEIstudyProgramme{{{study_programme}}}
\\FEIstudyProgrammeEn{{{study_programme_en}}}
\\FEIstudyField{{{study_field}}}
\\FEIstudyFieldEn{{{study_field_en}}}
\\FEItrainingWorkplace{{{training_workplace}}}
\\FEItrainingWorkplaceEn{{{training_workplace_en}}}

% Supervisor
\\FEIsupervisor{{{supervisor}}}
"""

    if consultant:
        tex_content += f"\n\\FEIconsultant{{{consultant}}}\n"

    tex_content += f"""

% Bibliography database file
\\bibliography{{includes/bibliography.bib}}

\\begin{{document}}

\\frontmatter
\\FEIpdfInfo
\\FEIcover
\\FEItitlePage
\\includepdf[pages=-]{{includes/assignment.pdf}}
\\FEIthanks{{includes/thanks}}
\\FEIabstract{{includes/abstract}}
\\FEIabstractEn{{includes/abstractEN}}
```

```
\\FEIcontent
\\FEImanualListOfGlossaries{{includes/manual_glossary}}
\\FEIlistOfAlgorithms
\\FEIlistOfListings

\\mainmatter
\\FEIintroduction{{includes/introduction}}
\\FEIcore{{includes/core}}
\\FEIconclusion{{includes/conclusion}}
\\FEIbibliography
\\FEIaiDeclaration{{includes/ai_declaration}}
\\backmatter
\\FEIappendix{{Algoritmus\\label{{att:algorithms}}}}{{includes/attachmentA}}
\\FEIappendix{{Výpis dlhého kódu\\label{{att:listings}}}}{{includes/attachmentB}}
\\FEIappendix{{Slovníček pojmov\\label{{att:dictionary}}}}{{includes/attachmentC}}

\\end{{document}}
"""

    with open("thesis.tex", "w", encoding="utf-8") as file:
        file.write(tex_content)

    print("Súbor thesis.tex bol úspešne vygenerovaný.")


# Testovacie volanie funkcie
generate_thesis_tex(
    author="RNDr. Juraj Chlpík, PhD.",
    reg_nr="FEI-xxxx-xxxx",
    title="Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme
    LaTeX",
    title_en="Extended thesis template at FEI STU in Bratislava in LaTeX system",
    day="31",
    month="12",
    year="2024",
    keywords="záverečná práca, šablóna, LaTeX, formátovanie textu, citácie",
    keywords_en="Final thesis, template, LaTeX, text formatting, citations",
    study_programme="názov študijného programu",
    study_programme_en="Study program in English",
    study_field="názov študijného odboru",
    study_field_en="Field of the study in English",
    training_workplace="Názov školiaceho pracoviska",
    training_workplace_en="Training Work place",
    supervisor="tituly Meno Priezvisko, tituly",
    consultant="",
```

```
    thesis_type="bp",  # Možnosti: "bp", "dp"
    language="sk",  # Možnosti: "sk" (slovenčina - predvolená), "en" (angličtina)
)
```

Výpis kódu 47: Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting`

Dodatok C: Slovníček pojmov

Abstrakt – Krátke zhrnutie hlavných bodov dokumentu, vrátane cieľov, metodológie a výsledkov.

Bod (pt) – Jednotka merania veľkosti písma; 1 bod sa rovná približne 0,352 mm.

Bold – Písmo, ktoré je tučné, používané na zvýraznenie dôležitých častí textu a v nadpisoch.

Citácia – Uvedenie zdroja informácií v texte, ktoré umožňuje identifikáciu pôvodného materiálu.

Font – Typ písma, ktorý sa používa na zobrazenie textu. Rôzne fonty majú odlišný štýl a vzhľad.

Graf – Vizualizácia údajov, ktorá zobrazuje vzťah medzi premennými.

Kapitola – Hlavná časť dokumentu, ktorá sa zaoberá konkrétnou témou a je zvyčajne označená číslom alebo názvom.

Korektúra – Proces revízie a opravy textu s cieľom odstrániť chyby.

Kurzíva – Typ písma, ktorý je sklonený doprava a používa sa na zvýraznenie textu, názvov alebo cudzích výrazov.

Metodológia – Opis metód a techník použitých pri výskume alebo analýze.

Normostrana – Základná jednotka pre meranie dĺžky textu, ktorá zvyčajne obsahuje 1 800 znakov vrátane medzier.

Obrázok – Grafický prvok alebo fotografia, ktorá ilustruje text.

Odsadenie – Vzdialenosť prvého riadku odseku od okraja stránky.

Odsek – Základná jednotka textu, ktorá obsahuje súvislé myšlienky. V LaTeX-u sa odseky oddeľujú prázdnym riadkom v zdrojovom texte.

Plagiátorstvo – Prezentovanie cudzej práce alebo myšlienok ako vlastných bez riadneho citovania.

Rez písma – Variant typu písma, ktorý určuje štýlové úpravy, ako sú normálny, tučný (bold), kurzíva (italic) alebo podčiarknutie.

Riadkovanie – Vzdialenosť medzi jednotlivými riadkami textu.

Sadzba – Proces usporiadania textu a grafiky na stránke.

Súbor – Elektronický dokument, ktorý obsahuje text alebo dáta.

Šablóna – Prednastavené formátovanie pre rôzne typy textu, ktoré zabezpečuje jednotný vzhľad.

Tabuľka – Usporiadanie údajov do riadkov a stĺpcov.

Titulný list – Prvá stránka dokumentu, ktorá obsahuje základné informácie, ako je názov práce a meno autora.

Typografia – Umenie a technika usporiadania textu tak, aby bol esteticky príťažlivý a čitateľný.

Úvod – Prvá časť dokumentu, ktorá predstavuje tému a ciele výskumu.

Vedecká práca – Akademický dokument, ktorý prezentuje výsledky výskumu, analýzy alebo diskusie o špecifickej téme.

Zdroje primárne a sekundárne – Primárne zdroje sú originálne dokumenty; sekundárne zdroje interpretujú alebo analyzujú primárne zdroje.